SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
FACULTY OF SCIENCE AND HUMANITIES
DEPARTMENT OF COMPUTER APPLICATIONS



INTERNSHIP REPORT

NAME	:	MATHESHWARAN R
REGISTER NUMBER	:	RA2532241010311
CLASS	:	MCA **SECTION:** F
YEAR & SEMESTER	:	II YEAR / III SEMESTER
ORGANIZATION NAME	:	Department of Computer Applications, SRM Institute of Science and Technology (In-house Internship)
DOMAIN	:	WEB DEVELOPMENT INTERNSHIP
COURSE CODE	:	PCA25P01L
DATE	:	25 May 2026 – 25 August 2026
DURATION	:	3 Months

BONAFIDE CERTIFICATE

This is to certify that the Internship report titled **"Web Development Internship on the CuriousBees Project at SRM Institute of Science and Technology"** is a bonafide work carried out by **MATHESHWARAN R** (Register Number: RA2532241010311) for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by this student.

INTERNAL EXAMINER 1 **INTERNAL EXAMINER 2**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the **Department of Computer Applications, SRM Institute of Science and Technology**, for providing me the opportunity to complete my internship within the institution and for entrusting our team with the development of the CuriousBees platform.

I am grateful to **Dr P J Arul Leena Rose**, Head of the Department, and **Dr Albert Antony Raj S**, Deputy Dean, for their support of this in-house internship. I am deeply thankful to my internship supervisor, **Ms. Simi Gracia Sunil Christopher**, Assistant Professor and Faculty Coordinator of the programme, for the continuous guidance, encouragement, and valuable insights provided throughout the internship.

I also extend my appreciation to my Professors at SRM Institute of Science and Technology for their cooperation and support. I am equally grateful to my academic mentor for their advice and assistance. I thank the members of the CuriousBees project team, whose work across the AI/ML, Backend, Frontend and UI/UX, DevOps, Full Stack and Android tracks made this platform possible. Finally, I thank my family and friends for their constant motivation during this internship.

MATHESHWARAN R
RA2532241010311

TABLE OF CONTENTS

INTERNSHIP COMPLETION CERTIFICATE 6

1. INTRODUCTION 7

1.1 Background of the Internship 7

1.2 Objectives of the Internship 8

1.3 Significance and Description of the Internship 9

1.4 Scope of the Report 10

1.5 Organization of the Report 10

2. COMPANY PROFILE 11

2.1 Overview of the Organization 11

2.2 Mission, Vision, and Values 12

2.3 Organizational Structure 13

2.4 Products or Services Offered 14

2.5 Internship Completion Certificate 15

3. ROLE AND RESPONSIBILITIES 16

3.1 Description of the Tasks and Projects 16

3.2 Tools and Technologies Used 19

3.3 Detailed Account of the Experience 20

3.4 Fundamentals of Full-Stack Web Architecture 22

3.5 Designing the REST API 23

3.6 Relational Data Modelling with Prisma 25

3.7 Authentication and Authorization 26

3.8 Secure File Storage with Presigned URLs 28

3.9 Applied Artificial Intelligence: Email-to-Event Extraction 29

3.10 Testing Strategy and Quality Assurance 31

3.11 Continuous Integration, Delivery and Cloud Deployment 32

3.12 Web Application Security Practices 34

3.13 Performance, Scalability and Operational Safety 34

3.14 Documentation, Version Control and Team Practices 35

4. DAILY/WEEKLY LOG OF ACTIVITIES 36

4.1 Weekly Log 36

4.2 Daily Log 38

4.3 Major Projects and Their Outcomes 40

4.4 Challenges Faced and How They Were Addressed 44

5. SKILLS ACQUIRED 46
5.1 Technical Skills 46
5.2 Soft Skills 47
5.3 New Knowledge and Competencies Developed 47
5.4 Analysis and Discussion 47
6. CONCLUSION 49
6.1 Summary of the Internship Experience 49
6.2 Influence of the Internship on Career Goals 49
REFERENCES 50
APPENDIX A: GLOSSARY OF TERMS 51
APPENDIX B: PROJECT STATISTICS AT DELIVERY 53

INTERNSHIP COMPLETION CERTIFICATE

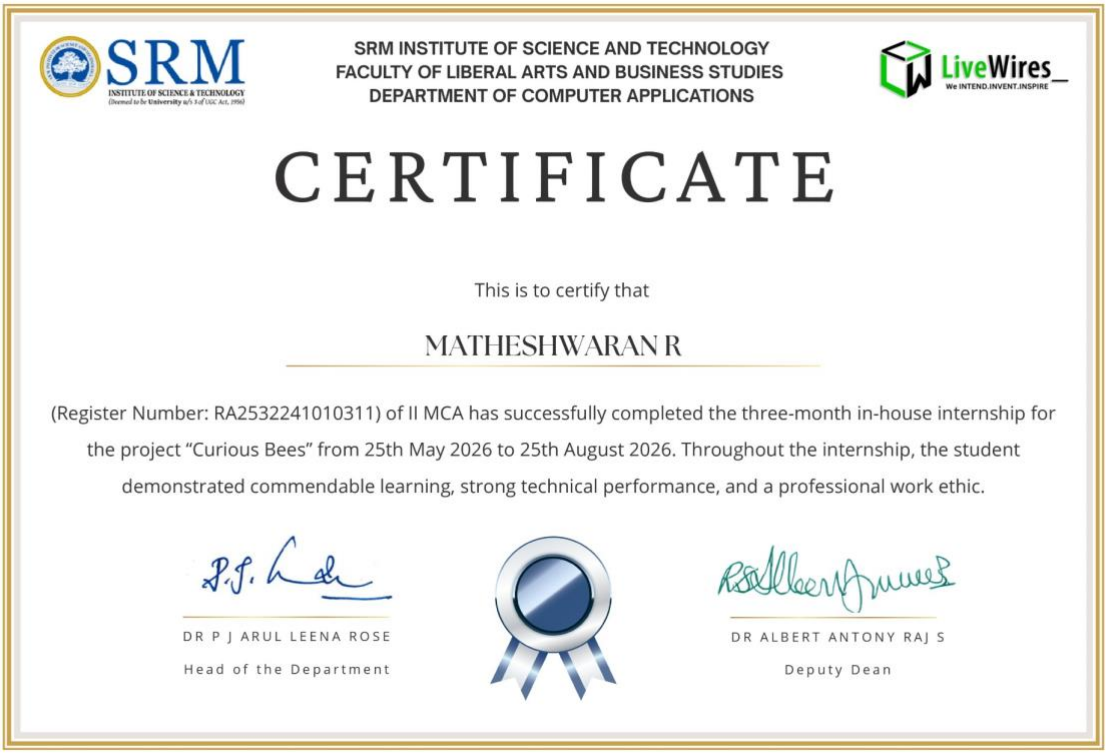


Figure 1.1: Internship Completion Certificate issued by the Department of Computer Applications, SRMIST (25 May – 25 August 2026)

1. INTRODUCTION

1.1 Background of the Internship

Research in a university depends on people working together. A doctoral scholar works with a supervisor for several years, reports progress at regular intervals, shares drafts and data, attends seminars, and collaborates with other researchers. In many institutions this work is coordinated through email threads, spreadsheets, messaging groups and paper forms. Information becomes scattered, the status of a scholar is difficult to follow, and announcements of events and opportunities reach only some of the people who need them.

A web platform can bring this work into one place: a single, secure application in which scholars, supervisors and administrators each see what is relevant to their role. Building such a platform is a full-stack web development problem. It needs a responsive user interface, a well-designed application programming interface (API), a relational database that mirrors the structure of the institution, secure authentication and file storage, integration with external services, automated testing, and a reliable path to deployment in the cloud.

The Department of Computer Applications of SRM Institute of Science and Technology (SRMIST) announced a Faculty-Guided Summer Internship Programme for 2026, beginning on 25 May 2026. It was announced as a 30-day programme with a provision for extension, and its focus was to build a real-time collaborative platform under faculty guidance, with students working in dedicated technical teams for UI/UX, Frontend Development, Backend Development, DevOps and AI/ML. The programme was coordinated by Ms. Simi Gracia Sunil Christopher, Assistant Professor, Department of Computer Applications. The internship was extended, and the completion certificate issued by the department records a **three-month in-house internship on the project "Curious Bees" from 25 May 2026 to 25 August 2026**.

I took part in the internship as a student of the Master of Computer Applications (MCA) programme, in my second year and third semester, under course code PCA25P01L. I served as the **Team Lead and Full Stack Developer** (Full Stack – DevOps track) of an eleven-member team. The project assigned to our team was **CuriousBees**, a research collaboration and governance platform for the institution. CuriousBees brings doctoral supervision, shared research workspaces, a research feed, research opportunities, an institutional events calendar and institutional administration into one product for three groups of users: research scholars, research supervisors and institute administrators.

The project started as a Phase 1 minimum viable product named SRM ReCollab, which I set up on 29 May 2026. It was later renamed ReCollab and finally CuriousBees. I first developed

the platform in my own repository and later moved the complete codebase, with its history, to the organization repository used for delivery. Team members developed their modules in separate repositories, and I integrated their work into the main repository. After the internship period, I completed the integration and delivered the final build on 4 October 2026. The application has been delivered to the institution, and its production deployment will be carried out by ITKM.

1.2 Objectives of the Internship

The main objective of the internship was to design, build and deliver a production-grade research collaboration platform for the institution, and in doing so to gain practical experience of full-stack web development and of leading a team. The specific objectives were as follows.

1. To design and build a production-grade, full-stack web platform for research collaboration at SRMIST, rather than a classroom prototype.
2. To understand and apply the architecture of a modern web application: user interface, API, database and cloud services.
3. To implement institutional governance with three clearly separated roles, administrator-provisioned accounts and an audit trail.
4. To model the institution faithfully in a relational database, from campus to faculty to department.
5. To integrate the services that research work depends on: secure file storage, transactional email, online meetings and search.
6. To apply artificial intelligence (AI) technology where it removes manual work, specifically to turn institutional event emails into calendar events without compromising accuracy.
7. To practise secure development against the most common web application vulnerabilities.
8. To adopt professional engineering practices: a TypeScript monorepo, shared type definitions, automated testing and continuous integration.
9. To prepare the platform for reliable deployment and operation in the cloud.
10. To lead a team through planning, task allocation, code review and integration.
11. To improve communication and documentation skills, so that the platform can be deployed and maintained by others.

12. To gain exposure to delivering software for a real institution within fixed timelines.

1.3 Significance and Description of the Internship

The internship was significant to me because it required delivering software for a real institution, with real constraints. Accounts had to be tied to institutional email domains, the academic structure had to match the university's faculties and departments, private research files had to stay private, and the system had to be maintainable by a different team after delivery. Working on a single product across its full life cycle, from the first commit to a deployment-ready build, exposed me to decisions that individual course assignments do not require, such as choosing an authentication provider, planning database migrations and designing for operational safety.

Figure 1.2 presents the high-level architecture of the platform that the internship produced. CuriousBees is organised as a TypeScript monorepo managed with npm workspaces. The web application (apps/web) is built with Next.js 15 and React 19 and provides 75 page routes covering the portals for each role, supervision, workspaces, the research feed and the administrative console. The REST API (apps/api) is built with NestJS 11 and exposes 253 HTTP endpoints across 40 controllers under the /api prefix, with interactive Swagger documentation. Shared packages keep the contract between the frontend and the backend consistent.

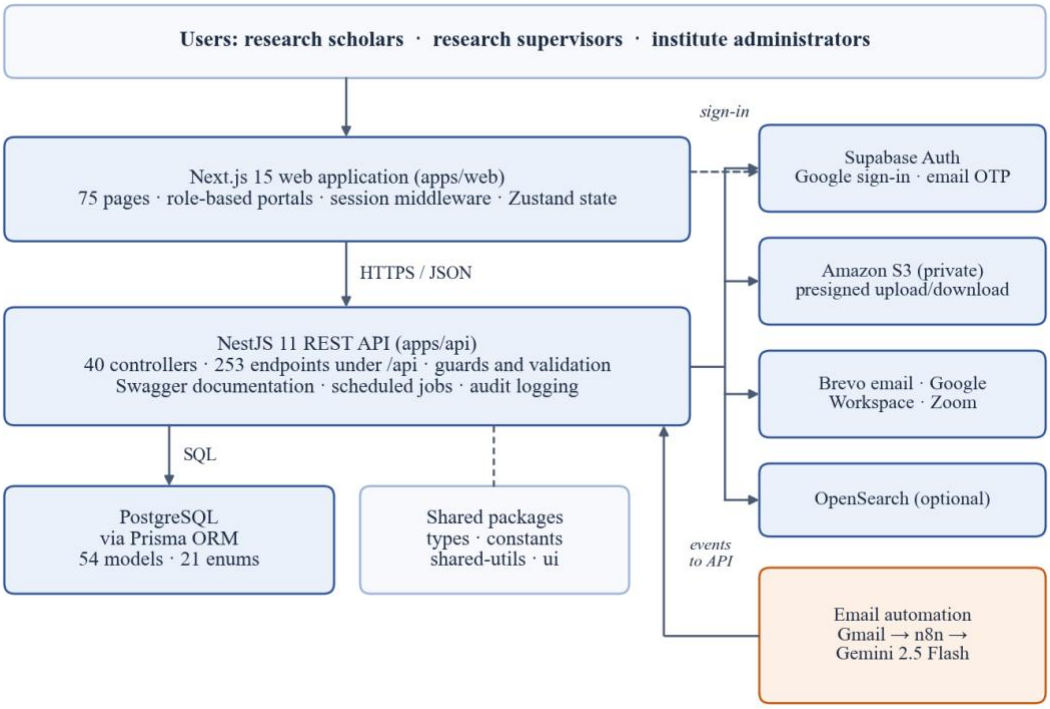


Figure 1.2: High-level system architecture of CuriousBees

Data is stored in PostgreSQL through the Prisma object-relational mapper (ORM), in a schema of 54 models and 21 enumerations. Authentication uses Supabase Auth with Google sign-in for allowed institutional email domains and an email one-time password (OTP) as a fallback, while authorization is enforced by the API itself. Research files are stored in a private Amazon S3 bucket and are accessed only through short-lived presigned URLs. The platform sends transactional email through Brevo, creates meetings through Google Workspace and Zoom, and can use OpenSearch for search, falling back to PostgreSQL queries when it is not configured. An n8n workflow that uses the Gemini 2.5 Flash model reads institutional event announcements and submits them to the API, which decides whether to publish them.

1.4 Scope of the Report

This report covers the three-month internship from 25 May 2026 to 25 August 2026, together with the completion and delivery work that followed until 4 October 2026 under the extension provision of the programme. It describes the organization, my role and responsibilities, the concepts and technologies applied, the weekly and daily activities, the challenges encountered and the skills developed.

The diagrams in this report were prepared to explain the design of the platform. Every chart is drawn from real data: the version-control history of the delivered repository and measurements of the codebase at the delivery commit. Where a worked illustration uses sample values, this is stated in the text. The screenshots show the delivered application. No passwords, keys or other credentials appear in this report.

1.5 Organization of the Report

The report is divided into six chapters. Chapter 1 introduces the internship and its objectives. Chapter 2 presents the profile of SRM Institute of Science and Technology, the Department of Computer Applications and the project team. Chapter 3 explains my role and responsibilities, the tools and technologies used, and the main web development concepts and techniques applied in the project. Chapter 4 provides the weekly and daily log of activities with the outcomes and challenges. Chapter 5 discusses the skills acquired, together with an analysis. Chapter 6 concludes the report with a summary and the influence of the internship on my career goals. References, a glossary of terms (Appendix A) and the project statistics at delivery (Appendix B) are provided at the end.

2. COMPANY PROFILE

The internship was an in-house, faculty-guided programme conducted within SRMIST. This chapter therefore profiles SRM Institute of Science and Technology and its Department of Computer Applications as the host organization, and then describes the CuriousBees project team in which I worked.

2.1 Overview of the Organization

SRM Institute of Science and Technology (SRMIST) is a deemed-to-be university under Section 3 of the UGC Act, 1956. The SRM group of educational institutions began with a primary school in 1968, and in 2002–2003 the Ministry of Education and the University Grants Commission approved the launch of SRMIST as a deemed university, with Dr T. R. Paarivendhar as its founder Chancellor. The university offers diploma, undergraduate, postgraduate, doctoral and post-doctoral programmes in Engineering and Technology, Medical and Paramedical sciences, Science and Humanities, Management, Law and Agricultural Sciences. Its strategic plan describes about 60,000 students and about 3,600 faculty members across five campuses. SRMIST holds the highest grade of A++ from the National Assessment and Accreditation Council (NAAC) and is placed in Category I by the Ministry of Education and the UGC (SRM Institute of Science and Technology [SRMIST], 2023). The main campus is at Kattankulathur, SRM Nagar, Chengalpattu District, Tamil Nadu.

The Department of Computer Applications has functioned since 2006 and offers the BCA and BCA (Data Science) programmes at the undergraduate level and the MCA and M.Sc. Applied Data Science programmes at the postgraduate level (SRMIST, n.d.). Our team itself included students of the MCA, MCA (Generative AI) and BCA programmes. The department conducted the Faculty-Guided Summer Internship Programme 2026 to give students practical, team-based development experience on a real project. The details of the organization and the internship are summarised in Table 2.1.

Table 2.1: Organization and Internship Details

Particular	**Details**
Organization	SRM Institute of Science and Technology (deemed to be university under Section 3 of the UGC Act, 1956)
Department	Department of Computer Applications
Faculty (as shown on the certificate)	Faculty of Liberal Arts and Business Studies

Particular	Details
Campus	Kattankulathur, SRM Nagar, Chengalpattu District, Tamil Nadu
Internship type	In-house, faculty-guided internship
Project	CuriousBees (“Curious Bees”) research collaboration and governance platform
My role	Team Lead and Full Stack Developer (Full Stack – DevOps track)
Internship period	25 May 2026 to 25 August 2026
Duration	3 Months
Faculty Coordinator	Ms. Simi Gracia Sunil Christopher, Assistant Professor
Certificate signatories	Dr P J Arul Leena Rose, Head of the Department; Dr Albert Antony Raj S, Deputy Dean
Final delivery	4 October 2026 (production deployment by ITKM)
Website	www.srmist.edu.in

2.2 Mission, Vision, and Values

Vision. The university’s stated vision is to emerge as a world-class university in creating and disseminating knowledge, and to give students a unique learning experience in science, technology, medicine, management and other areas of scholarship that serves the world and the betterment of humanity (SRMIST, 2023).

Mission. The mission of the university has three parts (SRMIST, 2023):

- **Move up** through international alliances and collaborative initiatives to achieve global excellence.
- **Accomplish a process** to advance knowledge in a rigorous academic and research environment.
- **Attract and build people** in a rewarding and inspiring environment that fosters freedom, empowerment, creativity and innovation.

Values. The core values set out in the university’s strategic plan are excellence, integrity and academic freedom; global vision and local commitment; and inclusiveness, diversity and empathy (SRMIST, 2023). The motto “Learn · Leap · Lead” appears in the university’s emblem.

These values were reflected in the way the internship was carried out. Within the project, our team worked to a set of practical values that followed from building software for an institution: protecting the privacy of research data, keeping every sensitive administrative

action auditable, never presenting invented information as fact, and documenting the system well enough for others to operate it.

2.3 Organizational Structure

The internship was organised as a faculty-guided team within the Department of Computer Applications. The Faculty Coordinator guided the project and reviewed its progress. As Team Lead, I coordinated the work of the technical tracks and was responsible for integrating their modules into the delivered product, in addition to my own full-stack and DevOps work. Figure 2.1 shows this structure, and Table 2.2 lists the members of the team with their programmes, sections and tracks.

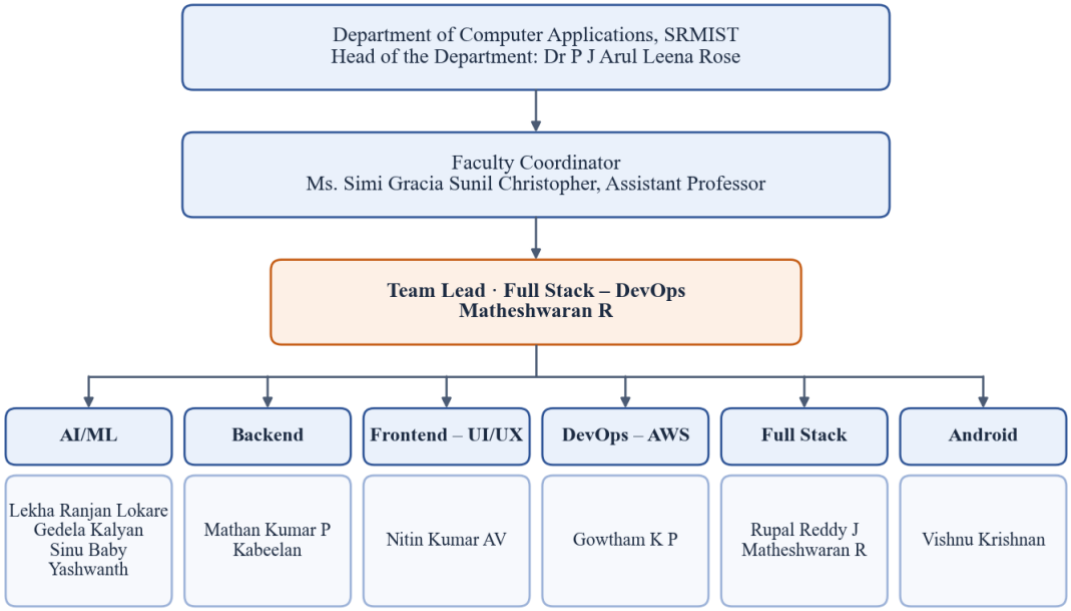


Figure 2.1: Organizational structure of the CuriousBees project team

Table 2.2: CuriousBees Project Team

S. No.	Name	Programme	Section	Team / Track
1	Lekha Ranjan Lokare	MCA (Generative AI)	D	AI/ML
2	Gedela Kalyan	MCA (Generative AI)	D	AI/ML
3	Sinu Baby	MCA (Generative AI)	B	AI/ML
4	Yashwanth	MCA (Generative AI)	D	AI/ML
5	Vishnu Krishnan	BCA	A	Android
6	Mathan Kumar P	MCA (Generative AI)	B	Backend
7	Kabeelan	MCA	B	Backend
8	Gowtham K P	MCA (Generative AI)	B	DevOps – AWS
9	Nitin Kumar AV	BCA	A	Frontend – UI/UX

S. No.	Name	Programme	Section	Team / Track
10	Rupal Reddy J	MCA (Generative AI)	B	Full Stack
11	Matheshwaran R	MCA	F	Full Stack – DevOps (Team Lead)

Each track owned one technical area of the platform, matching the teams named in the internship announcement, with an additional Android track. Work flowed in both directions: I allocated modules and defined the interfaces between them, the tracks built and tested their modules, and I reviewed the contributions before integrating them into the delivery repository. This structure gave me a clear understanding of how responsibilities are shared in a software team, how technical decisions are communicated, and how work is reviewed before it is considered complete.

2.4 Products or Services Offered

As an institution of higher education, SRMIST offers diploma, undergraduate, postgraduate, doctoral and post-doctoral programmes across its faculties and supports research in each discipline (SRMIST, 2023). The Department of Computer Applications offers undergraduate and postgraduate programmes in computer applications and data science (SRMIST, n.d.).

The product that our project team delivered to the institution is the CuriousBees platform itself. Table 2.3 summarises what it offers to each of its three user roles. Institute administrators deliberately do not take part in research activities. A dedicated guard in the API, the ResearchParticipantGuard, restricts research features to scholars and supervisors, while administrators govern accounts, institutional structure, moderation and auditing.

Table 2.3: Capabilities of CuriousBees by User Role

Role	Capabilities provided by the platform
Research Scholar	• Onboarding with faculty and department selection • Supervision requests to supervisors in the same department (up to six scholars per supervisor) • My Research: thesis topic, research stage, milestones and progress reports • Research workspaces with files, milestones, updates and meetings • Research feed, comments, follows and collaboration requests in Curious Nexus • Research opportunities (viewing, posting and join requests), events calendar and notifications
Research Supervisor	• Supervision Panel to accept or decline requests and follow assigned scholars • Review of progress reports (on track, more information needed, delayed) • Creation and ownership of research workspaces for their scholars • Posting of research opportunities and calendar events
Institute Administrator	• Administrator-created accounts, with single and bulk (spreadsheet) provisioning • Faculties, departments and campuses; roles and permissions • Moderation queue, posts and announcements • Institutional events, and the Event Inbox for events extracted from email • Email delivery statistics, analytics, audit centre and system settings

2.5 Internship Completion Certificate

On completion of the internship, the Department of Computer Applications issued an Internship Completion Certificate, a copy of which is included in this report as Figure 1.1. The certificate confirms that I, Matheshwaran R (Register Number RA2532241010311) of II MCA, successfully completed the three-month in-house internship for the project “Curious Bees” from 25 May 2026 to 25 August 2026. It records that I demonstrated commendable learning, strong technical performance and a professional work ethic throughout the internship. The certificate is signed by Dr P J Arul Leena Rose, Head of the Department, and Dr Albert Antony Raj S, Deputy Dean, and carries the SRM emblem and the LiveWires emblem with the tagline “We Intend. Invent. Inspire.”

This internship was an important step in my academic and professional development. It showed me that practical work on a real product is the best way to apply classroom knowledge, and it encouraged me to keep improving my skills in full-stack development, cloud deployment and software quality.

3. ROLE AND RESPONSIBILITIES

3.1 Description of the Tasks and Projects

As Team Lead and Full Stack Developer, my role combined technical leadership of an eleven-member team with hands-on development across the whole platform. My main responsibilities were:

- Planning the project in phases and allocating modules to the technical tracks.
- Designing the monorepo, the API structure, the database schema and the shared type definitions.
- Implementing large parts of the web application and the API myself.
- Reviewing the modules contributed by team members and integrating them into the delivery repository.
- Designing and building the AI-assisted email-to-event ingestion pipeline with the AI/ML track.
- Preparing deployments, the continuous integration pipeline and the cloud deployment architecture with the DevOps track.
- Writing the documentation needed to deploy and maintain the platform after delivery.

The general process that the team followed for each module is shown in Figure 3.1. Each stage depended on the previous one: if a module was not reviewed carefully, integration became harder, and if the integrated system was not tested automatically, a change in one module could silently break another.

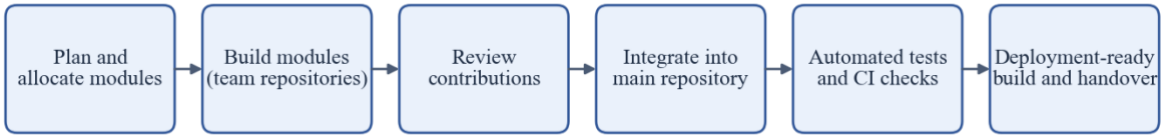


Figure 3.1: Development workflow followed in the project

3.1.1 Team Leadership and Coordination

- **Planning.** I broke the project into phases: a Phase 1 minimum viable product in the first week, consolidation of the architecture and core portals, feature expansion, a Phase 2 migration to a canonical institutional hierarchy, and finally production readiness. Each phase was documented in the repository.
- **Task allocation.** I assigned modules to the AI/ML, Backend, Frontend and UI/UX, DevOps, Full Stack and Android tracks according to the skills of each member, and defined the interfaces between modules so that they could be built in parallel.

- **Code review.** I reviewed the modules contributed by team members before integrating them, checking them for correctness, consistency with the shared types and adherence to the role-based access rules.
- **Coordination across tracks.** I aligned frontend and backend work through the shared packages (@curiousbees/types and @curiousbees/constants), coordinated deployment work with the DevOps track, and worked with the AI/ML track on the email-to-event automation.
- **Integration.** Team members developed their modules in separate repositories. I integrated these contributions into the main repository, resolved the conflicts between them and produced the final delivered build.
- **Documentation and conventions.** I maintained the README and a documentation portal of 40 documents covering architecture, database, deployment, security, the design system, setup guides for macOS, Linux and Windows, and a release checklist. I used the Conventional Commits format throughout the repository history (Figure 3.11).

3.1.2 Full Stack Development

I designed the monorepo and implemented large parts of both applications. On the backend, I organised the NestJS API into feature modules, each with its own controller, service and data-transfer objects. Table 3.1 lists the principal modules and their responsibilities.

Table 3.1: Principal Backend (API) Modules and Their Responsibilities

Module	Responsibility
auth, users, onboarding	Session verification, current-user profile, onboarding of scholars and supervisors
admin	Account provisioning (single and bulk), roles, user status, institution settings, safety rules
faculties, departments, research-domains	Institutional hierarchy and research taxonomy reference data
supervisor-requests, supervisors	Supervision requests, approvals, capacity limits and reassignment
my-research, reports, publications	Research record, progress reports and publications
workspaces, files	Shared workspaces, members, milestones and updates; S3 uploads and downloads
threads, comments, feed, collaborations	Research feed, nested comments, likes, saves, shares and collaboration requests
opportunities, events, announcements	Opportunities, the events calendar and system announcements
notifications	In-app notifications, Brevo email and the daily event digest

Module	Responsibility
search	Search through OpenSearch with a PostgreSQL fallback
integrations	Google Workspace (Meet, Chat spaces) and Zoom meeting providers
event-ingestion	Endpoints for the n8n workflow and the administrator review queue

I also implemented or integrated the following features:

- **Supervision workflow:** scholars request a supervisor in their department. The API enforces a capacity of six scholars per supervisor, and Brevo email alerts announce requests, approvals and rejections.
- **Workspaces and files:** uploads of up to 50 MB go directly from the browser to a private S3 bucket through presigned URLs that expire after five minutes; downloads use equally short-lived links (Section 3.8).
- **Research feed and Curious Nexus:** posts, papers and questions with multi-level comments, likes, saves, shares and reports, plus a hub for collaboration conversations.
- **Notifications:** in-app notifications, transactional email and a daily 8:00 AM (IST) event digest that is serialised with a PostgreSQL advisory lock, so that it runs only once even when several API instances are running.
- **Governance console:** user provisioning with spreadsheet import, faculties and departments, moderation, announcements, email delivery statistics, analytics, the audit centre and the Event Inbox.
- **Frontend:** the role-based portal structure in the Next.js App Router, route protection that sends each role to its own area, and many of the pages and shared components. In the final phase, I led a platform redesign that introduced a documented design system with light and dark themes, reduced-motion support and shared components.

3.1.3 AI-Driven Email-to-Event Ingestion

Institutional events at SRMIST are announced by email. To place them on the CuriousBees calendar without manual data entry, I designed and built an email-to-event ingestion pipeline with the AI/ML track. An n8n workflow reads the dedicated mailbox, filters the messages with deterministic rules, asks the Gemini 2.5 Flash model to extract the event details in a strict format, and submits the result to the CuriousBees API, which decides whether to publish the event, send it to an administrator for review, or record that it is not an event. The design and the reasons behind it are explained in Section 3.9.

In the first week of the project, the AI work had followed a different direction. A prototype extracted events from emails with a locally hosted large language model run through Ollama, and an analytics service grouped events by semantic similarity using pgvector embeddings and cosine similarity. These prototypes were removed during the architecture restructuring of 4 June 2026. The final design replaced them with the hosted Gemini model behind strict validation, which proved more reliable and simpler to operate.

3.1.4 DevOps, Deployment and Quality Assurance

I prepared the platform for deployment several times as the project evolved: on Vercel as serverless functions in the first week, on Railway in June, and finally as a complete architecture for Amazon Web Services (AWS) with container images, infrastructure templates, least-privilege access policies and a continuous integration and delivery pipeline in GitHub Actions (Section 3.11). I also built the automated test suites and the safeguards that keep tests away from real data (Section 3.10). At delivery, the automated test suites comprised 270 tests, all passing.

3.2 Tools and Technologies Used

The internship used a modern TypeScript stack throughout, so that the same language, types and tooling could be shared by the web application, the API, the scripts and the tests (Microsoft, n.d.). Table 3.2 lists the tools and technologies I used, with the versions recorded in the project's dependency manifests at delivery. The official documentation of each technology, listed in the References, was my primary learning resource throughout the internship.

Table 3.2: Tools and Technologies Used in the CuriousBees Project

Category	Technology (version)	Purpose in CuriousBees
Language and runtime	TypeScript 5.9; Node.js 22	One language across frontend, backend, scripts and tests
Frontend framework	Next.js 15.5 (App Router); React 19.2	Role-based portal, 75 page routes, routing and rendering
UI and styling	Tailwind CSS 3.4; Framer Motion 11; Lucide icons	Design system, light and dark themes, motion
Client state and forms	Zustand 4.5; TanStack Query 5; React Hook Form 7; Zod 3	Application state, server data, validated forms
Backend framework	NestJS 11.1 on Express	Modular REST API, guards, scheduling, Swagger documentation
API hardening	Helmet 7; @nestjs/throttler 6; class-validator; Winston	Security headers, rate limiting, input validation, structured logs

Category	Technology (version)	Purpose in CuriousBees
Database and ORM	PostgreSQL; Prisma 6.19	Relational data model (54 models) and migrations
Authentication	Supabase Auth (Google sign-in, email OTP)	Institutional sign-in (earlier Firebase and Clerk)
File storage	Amazon S3; AWS SDK v3 presigner	Private research files with presigned URLs
Email and meetings	Brevo API; Google Workspace; Zoom	Transactional email; Meet and Chat spaces; Zoom meetings
Search	Amazon OpenSearch (optional)	Full-text search with a PostgreSQL fallback
AI and automation	n8n; Google Gemini 2.5 Flash; Gmail API	Email-to-event extraction with structured output
AI prototypes (Phase 1)	Ollama (local LLM); pgvector	Early event extraction and semantic clustering
Testing	Jest 30; Supertest 7; Node.js test runner	Unit, integration and workflow tests
Containers	Docker; Docker Compose	Local services, test infrastructure and production images
CI/CD and cloud	GitHub Actions; AWS ECS Fargate, ECR, RDS, CloudFront, IAM	Automated validation and the prepared deployment pipeline
Earlier hosting	Vercel; Railway	First deployments during the internship period
Version control	Git; GitHub	Monorepo with npm workspaces; 129 commits to delivery

The external services each had a clear and limited responsibility. Supabase proved who a user was, Amazon S3 stored files, Brevo delivered email (Brevo, n.d.), and Google Workspace and Zoom created meetings and collaboration spaces (Google for Developers, n.d.; Zoom Video Communications, n.d.). Keeping each service behind a small module in the API meant that a service could be replaced, or switched off in development and tests, without changing the rest of the code.

3.3 Detailed Account of the Experience

The internship began on 25 May 2026. In the first week, I focused on giving the team a working foundation as quickly as possible. I created the monorepo with a Next.js web application, a NestJS API and shared TypeScript packages, connected it to a Supabase PostgreSQL database and deployed the first build to Vercel. Most of my time in that first week went into making the deployment reliable, because serverless hosting of a Prisma-based

NestJS application required changes to the Prisma binary targets, the routing configuration and the API's global prefix. By the end of the week, the team had a deployable minimum viable product with sign-in and a first AI prototype.

The second week was the most intensive of the internship, with 44 commits. Having seen the first prototype, I restructured the codebase into modular feature folders. That made it possible to divide work across the tracks and to build workspaces, the opportunities portal, the administrative dashboard and the onboarding flow in parallel. I also learned an important lesson about scope: I introduced Redis and BullMQ queues for notifications and audited their availability, then removed them within the same week, because they added operational complexity that the platform did not need at its scale. The same week, I moved authentication from Firebase to Clerk and hardened the API for deployment on Railway.

In the third, fourth and fifth weeks, the platform gained its governance console and its social features. I built the institute administration modules and consolidated them into a single user-management structure, made the public pages accessible, and added configurable allowed email domains. The research feed, with discovery, nested comments, saved posts, sharing and sorting, took shape during these weeks.

From the end of June to the beginning of August, only one change was committed to the delivery repository, a fix to the API's network binding for deployment on 13 July. The modules contributed by team members were developed in separate repositories and were integrated into the delivery repository later. Work in the delivery repository resumed strongly on 10 August. In the remaining weeks of the internship, I built the modular events and opportunities dashboards, the Curious Nexus hub, the researcher directory, multi-level threaded comments and collaboration requests. I then made the decision to migrate authentication to Supabase, added an institutional email OTP fallback, implemented the integrations module for Google Workspace and Zoom, and built the supervision request management with scholar reassignment and a standard capacity of six scholars per supervisor. On the last day of the internship, I removed the socket-based chat module in favour of a simpler design, a pattern that recurred throughout the project: choosing fewer moving parts once the real requirements were clear.

After the internship period, I continued under the extension provision until delivery. The final phase, from 28 September to 4 October 2026, focused on delivery. I completed the Phase 2 migration to a canonical institutional hierarchy with test coverage, prepared the AWS deployment infrastructure and the CI/CD pipeline, added administrator safety rules and root-

administrator protection, led the platform redesign, added single and bulk user provisioning, integrated Google Workspace Chat spaces for research workspaces, and built the email-to-event ingestion pipeline with its review queue. Throughout this phase, I integrated the modules delivered by my team members and prepared the documentation that ITKM will use to deploy the platform.

The following sections describe the main web development concepts and techniques that I applied, each illustrated with how it was used in CuriousBees.

3.4 Fundamentals of Full-Stack Web Architecture

3.4.1 The three tiers of a web application

A web application is usually divided into three tiers. The **presentation tier** runs in the user's browser and displays information. The **application tier** runs on a server, receives requests, applies business rules and decides what each user may do. The **data tier** stores information permanently. In CuriousBees, the presentation tier is the Next.js web application, the application tier is the NestJS API, and the data tier is PostgreSQL (Figure 1.2). Keeping the tiers separate means that each can be developed, tested, secured and scaled on its own.

3.4.2 Monorepo and shared contracts

A monorepo keeps several related applications in one repository. CuriousBees uses npm workspaces to hold the web application, the API and four shared packages. The package @curiousbees/types defines the shapes of the data exchanged between the frontend and the API, and @curiousbees/constants holds values such as role names and limits. Because both applications import the same definitions, a change to a data shape is caught by the TypeScript compiler in every place that uses it, instead of failing at run time in the browser. For a team working in parallel, these shared packages acted as the contract between modules.

3.4.3 Rendering and routing in Next.js

Next.js builds on React, a library for composing user interfaces from components (Meta Open Source, n.d.). The App Router maps folders to URLs and allows layouts to be shared between pages (Vercel, n.d.). CuriousBees uses route groups to give each role its own portal under a shared layout, and Next.js middleware to refresh the user's session before a protected page is rendered. Interactive pages are client components that fetch data from the API, while the build produces a standalone server that can run in a container. Route protection in the browser is only a convenience for users; every permission is checked again by the API.

3.5 Designing the REST API

3.5.1 Resources, routes and HTTP methods

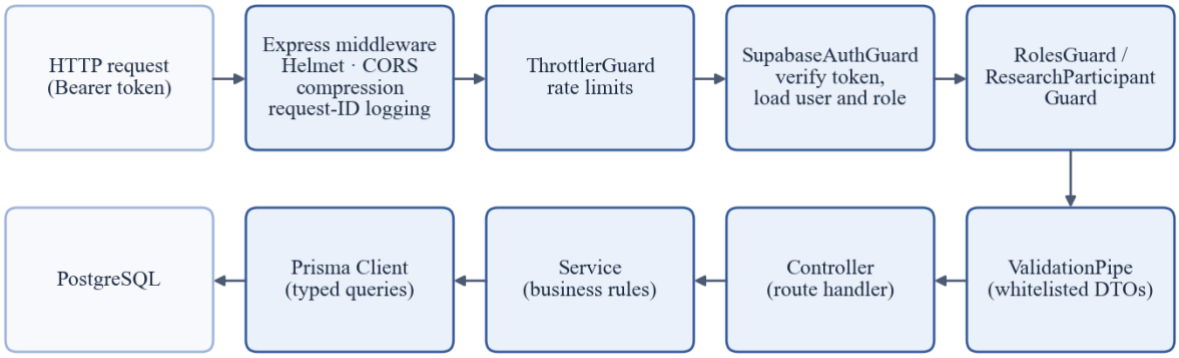
Representational State Transfer (REST) is an architectural style in which a server exposes resources that clients manipulate through a uniform interface (Fielding, 2000). In practice, each resource has a URL, and the HTTP method states the intended action: GET reads, POST creates or triggers an action, PUT and PATCH update, and DELETE removes. Table 3.3 shows a selection of real endpoints from the CuriousBees API.

Table 3.3: Selected CuriousBees API Endpoints

Method	Route	Purpose	Notes
GET	/api/threads?limit=20&cursor=…	One page of the research feed	Cursor pagination; at most 50 items
POST	/api/threads	Create a research post	Scholars and supervisors only
POST	/api/threads/:id/like	Like or unlike a post	Toggles the current user's like
GET	/api/users/researchers	Researcher directory	Filters by role, department and interest
POST	/api/supervisor-requests	Request a supervisor	Capacity of six scholars checked
POST	/api/threads/files/upload-url	Get a presigned upload URL	Type, size and extension validated
PUT	/api/notifications/read-all	Mark notifications as read	Applies to the current user only
GET	/api/admin/event-ingestion	Event Inbox	Institute administrators only
POST	/api/integrations/n8n/events	Submit an extracted event	Machine token, not a user session
GET	/api/health/live	Liveness probe	Does not touch the database

3.5.2 Modules, controllers and services

NestJS organises an application into modules that group a controller, which maps routes to handler methods, and one or more services, which hold the business rules (NestJS, n.d.). Dependencies such as the database client are supplied by dependency injection, which makes each service easy to test with a substitute. Figure 3.2 shows the path of a request through the CuriousBees API, and Figure 3.3 shows how the 253 endpoints are distributed across the modules. The administration module is the largest, because governance of accounts and the institution needs many operations.



Any error on the way is turned into one JSON response with a request ID by the global exception filter (details hidden for 5xx in production).

Figure 3.2: Path of a request through the CuriousBees API

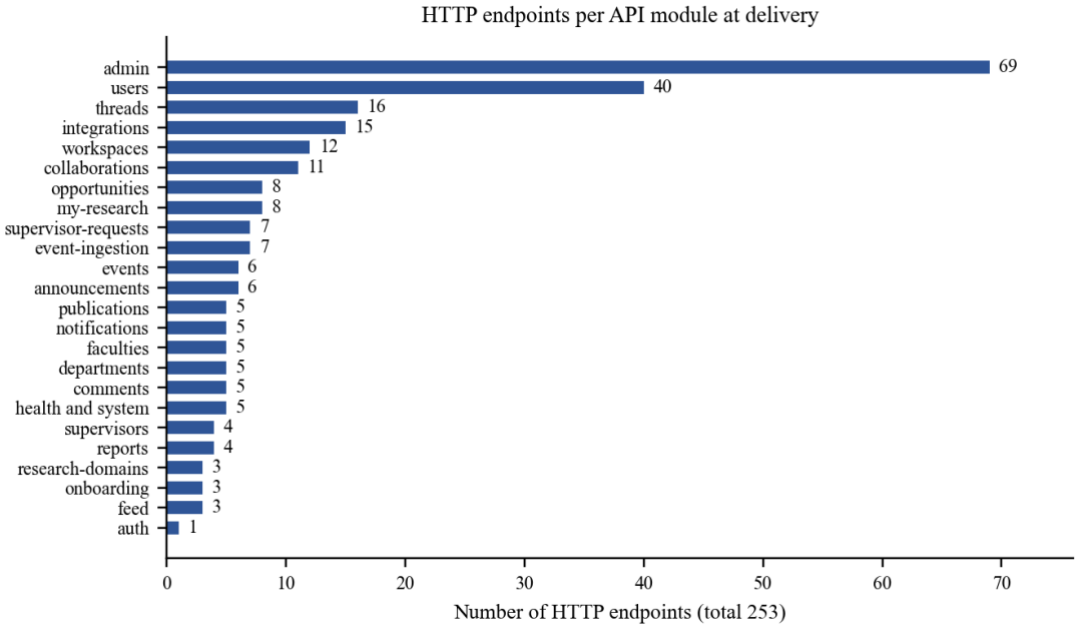


Figure 3.3: HTTP endpoints per API module at delivery (measured from the source code)

3.5.3 Validation, errors and pagination

Every request body is validated before it reaches a handler. A global validation pipe accepts only the properties declared in each data-transfer object and rejects requests that contain anything else, which prevents a client from setting fields it should not control, such as its own role. A global exception filter converts every error into a single JSON format with a request identifier, and hides internal details of server errors in production. Lists that can grow, such as the research feed and publications, use cursor-based pagination capped at 50 items per page, so that no request can ask the server for an unbounded amount of data. The API is documented interactively with Swagger at /api/docs during development.

3.6 Relational Data Modelling with Prisma

3.6.1 Entities and relationships

A relational database stores data in tables linked by keys, and enforces those links so that, for example, a progress report cannot refer to a scholar who does not exist (PostgreSQL Global Development Group, n.d.). Prisma describes the tables in a single schema file and generates a fully typed client from it (Prisma Data, n.d.). Figure 3.4 shows the core entities of the CuriousBees schema. The institutional hierarchy of campus, faculty and department anchors every user, so that supervision requests, events and opportunities can be validated against the correct department. Scholar and supervisor details live in one-to-one profile tables. Workspaces, their members and their files form a separate group, in which files are stored in S3 by object key only, never by public URL.

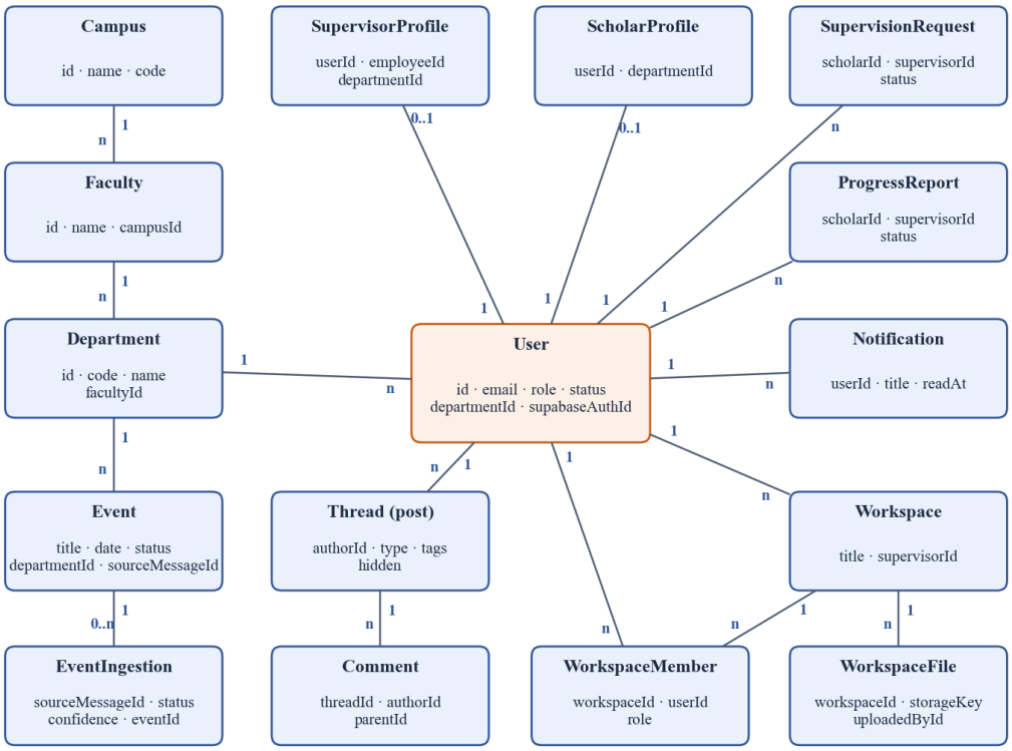


Figure 3.4: Core entities of the CuriousBees data model (simplified)

3.6.2 Normalising the institutional hierarchy – a worked illustration

In the early versions of the platform, a user's department was stored as free text. Free text is easy to collect but difficult to rely on: the same department can be written in several ways, and a department name alone does not say which faculty it belongs to. The Phase 2 migration replaced this with references to canonical records. Table 3.4 and Table 3.5 illustrate the idea with sample values invented only for explanation.

Table 3.4: Sample Free-Text Affiliations Before Normalisation

User	department (free text)	faculty (free text)
U1	Computer Applications	Science and Humanities
U2	Dept. of Computer Applications	—
U3	computer applications	FSH
U4	MCA Department	Science & Humanities

Table 3.5: The Same Affiliations After Normalisation

User	departmentId → Department	Faculty (through the department)
U1	Computer Applications (code MCA)	Derived from the department record
U2	Computer Applications (code MCA)	Derived from the department record
U3	Computer Applications (code MCA)	Derived from the department record
U4	Computer Applications (code MCA)	Derived from the department record

After normalisation, the four spellings refer to one department record, and the faculty is no longer stored separately for each user but derived through the department. A filter such as "all supervisors in this department" now returns every matching user, and renaming a department changes it everywhere at once. This is the practical benefit of normalisation described in the database literature (Kleppmann, 2017).

3.6.3 Migrations and schema evolution

The schema grew from a handful of tables to 54 models during the project. Every change to a production database must be made by a migration: a versioned script that moves the schema forward. I consolidated the early migration history into a single production baseline and made every later change a forward-only migration applied with prisma migrate deploy, which never drops data. The continuous integration pipeline applies all migrations to a fresh database and checks that the result matches the schema exactly, so that the code and the database can never drift apart unnoticed. Indexes were added for the columns used to filter and sort the largest lists, such as posts by visibility and date and notifications by user and date.

3.7 Authentication and Authorization

3.7.1 Proving identity and granting permission

Authentication answers the question "who is this person?", while authorization answers "what may this person do?". CuriousBees deliberately keeps these apart. Supabase Auth proves identity through Google sign-in or a one-time password sent by email (Supabase, n.d.). The CuriousBees API then decides what the person may do, using only its own

database. Accounts are created by institute administrators, sign-in is limited to allowed institutional email domains, and role or approval values sent by the browser are never trusted. This separation allowed the authentication provider to change twice during the project without rewriting a single access rule.

3.7.2 The sign-in and request flow

Figure 3.5 shows what happens when a signed-in user calls the API. The browser sends the Supabase access token with every request. A guard verifies the token, loads the user record by its identity, checks the email domain and the account status, and attaches the user's role. Role guards then allow or refuse the request. A permanent root administrator account is protected in code against demotion, suspension or deletion.

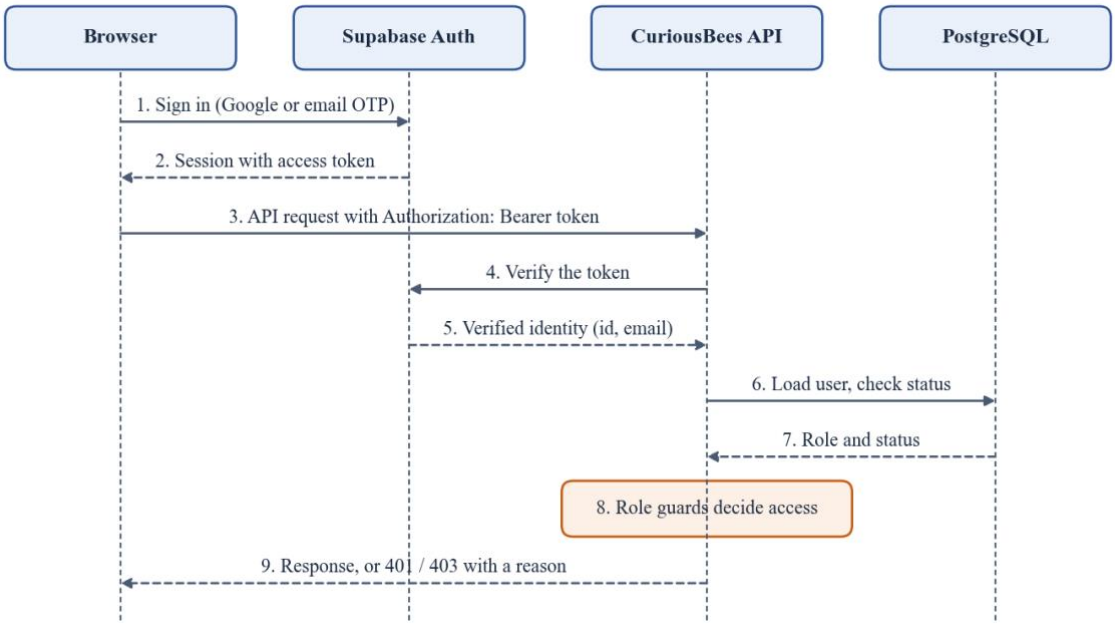


Figure 3.5: Authentication and authorization flow for an API request

3.7.3 Role-based access control

Role-based access control (RBAC) grants permissions to roles rather than to individual people. Table 3.6 summarises the main permissions of the three roles in CuriousBees. Each rule is enforced on the server by a guard, whatever the browser displays.

Table 3.6: Role-Based Access Control in CuriousBees

Action	Scholar	Supervisor	Administrator
Sign in with an allowed domain and an administrator-created account	Yes	Yes	Yes
Request a supervisor; submit progress reports	Yes	—	—
Accept or decline supervision requests; review reports	—	Yes	—

Action	Scholar	Supervisor	Administrator
Create research workspaces (the creator becomes the owner)	Yes	Yes	—
Post, comment and follow in the research feed	Yes	Yes	—
Post research opportunities	Yes	Yes	—
Post events on the calendar	—	Yes	Yes
Provision accounts; manage faculties and departments	—	—	Yes
Moderate content; view the audit centre and analytics	—	—	Yes
Review events extracted from email (Event Inbox)	—	—	Yes

3.8 Secure File Storage with Presigned URLs

Research files such as thesis drafts and data must remain private to the members of a workspace. Passing large files through the API would also consume its memory and bandwidth. CuriousBees solves both problems with presigned URLs: short-lived links, signed by the server, that allow the browser to upload or download one specific object directly from a private Amazon S3 bucket (Amazon Web Services, n.d.). Figure 3.6 shows the upload flow, and Table 3.7 lists the rules that the API enforces.

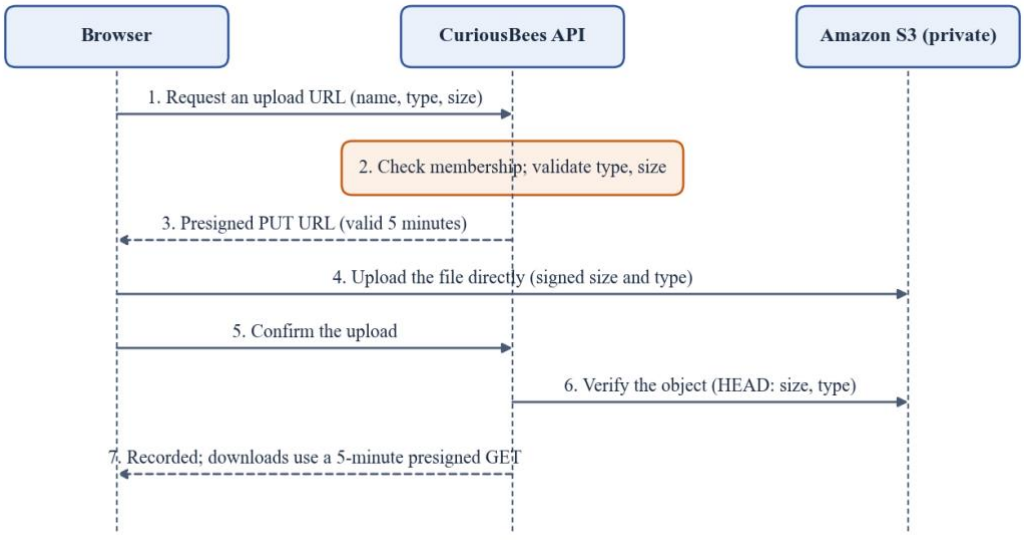


Figure 3.6: Direct upload to private storage with a presigned URL

Table 3.7: File Upload Rules Enforced by the API

Rule	Value in CuriousBees
Who may upload	Members of the workspace (checked before any URL is issued)
Allowed types	PDF, Word, PowerPoint, Excel, ZIP, JPEG/PNG/WebP images, text and CSV
Blocked names	Executable and script extensions anywhere in the name; disguised double extensions
Maximum size	50 MB, signed into the upload URL so that S3 rejects any other size
URL lifetime	5 minutes for uploads and for downloads
Object key	Built by the server from the workspace, user, a random identifier and a cleaned file name
After upload	The API checks the stored object's real size and type and deletes it if they do not match
Bucket	Private, with public access blocked, encryption at rest and HTTPS-only access

3.9 Applied Artificial Intelligence: Email-to-Event Extraction

3.9.1 The problem and the pipeline

Institutional event announcements arrive as emails written in many styles. A large language model (LLM) can read such an email and extract the title, date, time, venue and links, but it can also misread a date, invent a link, or mistake a newsletter for an event. Sending every email to a paid model would also be costly. The pipeline in Figure 3.7 therefore uses the model only where it adds value, and checks everything it produces. The workflow runs in n8n, an automation platform that connects services through configurable nodes (n8n GmbH, n.d.).

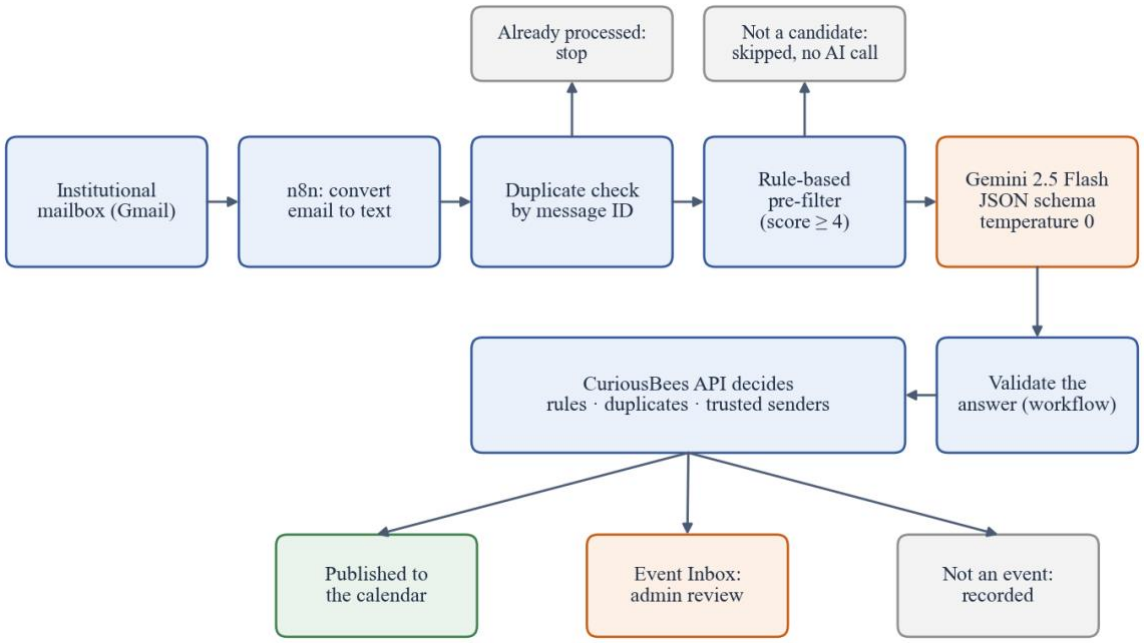


Figure 3.7: AI-assisted email-to-event ingestion pipeline

3.9.2 Rules before the model

Each email is first converted into plain text, keeping every link next to its label, and checked against the API by its unique Gmail message identifier, so that an email is never processed twice. A rule-based pre-filter then scores the email. It awards points for event words, dates, times, venues, meeting links, registration forms, calendar invitations and trusted senders, and subtracts points for invoices, promotions, newsletters and no-reply senders. Password resets, one-time passwords, bounces and automatic replies are always rejected. Only emails that reach a score of at least 4 are sent to the model, so most messages never incur an AI cost.

3.9.3 Structured output and deterministic settings

The request to Gemini 2.5 Flash asks for an answer in a strict JSON schema, with the temperature set to 0, the output limited to 1,024 tokens and reasoning tokens disabled (Google AI for Developers, n.d.). A schema turns the model's answer into data that a program can validate field by field, and a temperature of 0 makes the answer as repeatable as possible. Together, these settings make the model behave like a component with a defined interface rather than like a conversational assistant.

3.9.4 Independent validation and confidence bands

The answer is validated in the workflow and again in the API, which makes the final decision. The model reports a confidence value between 0 and 1, and the API places each candidate in one of three bands (Figure 3.8 and Table 3.8). Links are accepted only if they appear in the original email, and a year that the model had to infer sends the event to review.

A second check detects the same event arriving in a different email, including events that were already posted manually. The workflow's own code is covered by tests that use recorded model responses, so the automated tests never call the paid service.

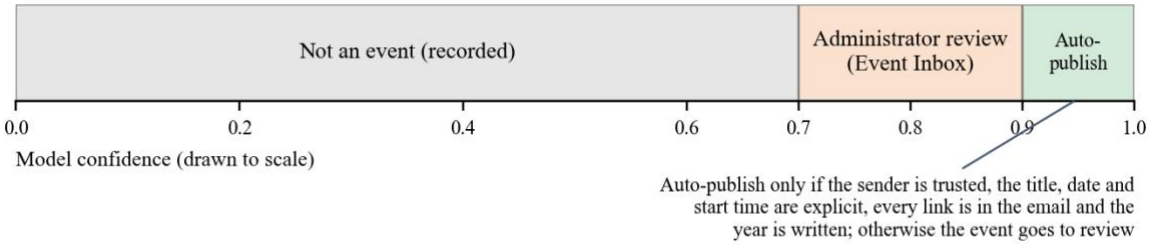


Figure 3.8: Decision bands for events extracted by the model

Table 3.8: Decision Rules for Extracted Events

Condition	**Outcome**
Confidence of at least 0.90, trusted sender, title, date and start time stated explicitly, every link present in the email, year written, plausible date	Published to the calendar automatically
Confidence from 0.70 to 0.89, or any missing or ambiguous detail	Sent to the administrators' Event Inbox for review
Confidence below 0.70	Recorded as not an event
Same Gmail message identifier seen before	Ignored, so retries never create duplicates
Same event already exists (from another email or posted manually)	Recorded as a duplicate

3.10 Testing Strategy and Quality Assurance

Automated tests turn the expected behaviour of a system into programs that run on every change. CuriousBees uses several kinds of tests, each with a different purpose (Table 3.9 and Figure 3.9). Unit tests check one service or guard at a time with the database replaced by a substitute, and run in seconds. Integration tests run the real HTTP pipeline against a real PostgreSQL database and an S3-compatible emulator, which catches problems that only appear when the parts work together. The test framework is Jest (OpenJS Foundation, n.d.).

Table 3.9: Automated Test Suites at Delivery

Suite	Tests	What it verifies
API unit tests (Jest)	172	Services, guards, validation and decision logic, with the database replaced
API integration tests (Jest and Supertest)	61	The real HTTP pipeline against a real PostgreSQL database and an S3 emulator
n8n workflow tests (Node.js test runner)	34	Pre-filter, request building, normalisation and the generated workflow file, with network access blocked
Web helper tests (Node.js test runner)	3	Safe rendering of links, meeting provider names and time zone labels
Total	**270**	**All passing at delivery**

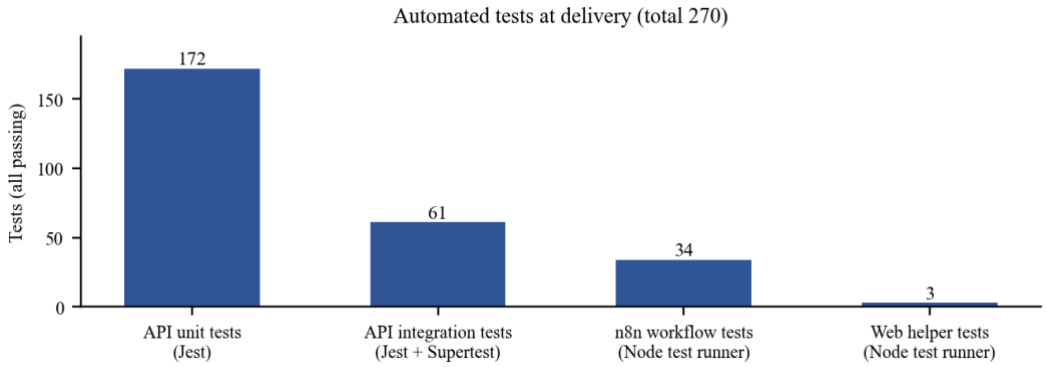


Figure 3.9: Automated tests at delivery by suite

Because the integration suite drops and recreates its database before running, it must never reach a real database. The suite runs only against one dedicated, disposable test database on the local machine, removes database and external-service credentials from its environment, and blocks every network connection that leaves the machine. Section 4.4 describes the incident that led to these safeguards.

3.11 Continuous Integration, Delivery and Cloud Deployment

Continuous integration means that every change is built and tested automatically, and continuous delivery means that a tested change can be released through a repeatable, automated process (Humble & Farley, 2010). The CuriousBees pipeline in GitHub Actions (Figure 3.10) runs linting, type checking, the unit, workflow and web tests, the migration drift check, the integration tests and the production builds on every change (GitHub, n.d.). Only then are container images built for the ARM64 architecture and pushed to a registry. After approval by a reviewer, a one-off task applies the database migrations, and the services are updated with a rolling deployment that keeps the platform available.

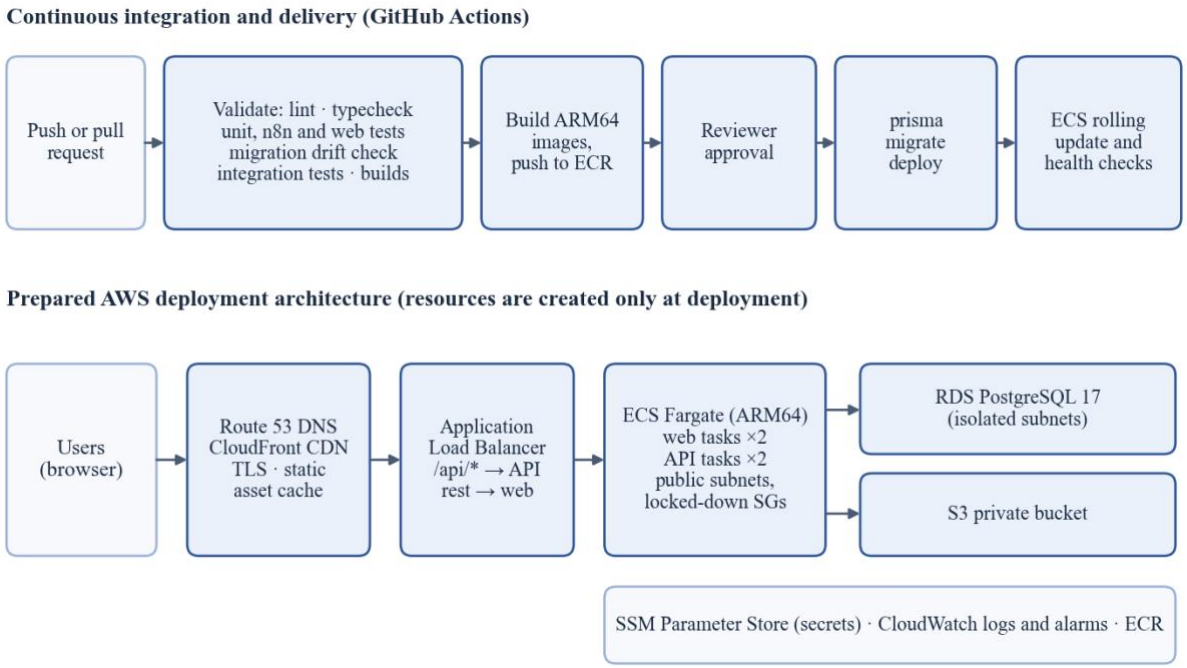


Figure 3.10: CI/CD pipeline and the prepared deployment architecture

The application runs in Docker containers, which package the code with its runtime so that it behaves the same on a laptop and in the cloud (Docker Inc., n.d.). I prepared a complete AWS architecture for production, summarised in Table 3.10. The resources in it are created only at the time of deployment, which ITKM will carry out.

Table 3.10: Components of the Prepared AWS Deployment Architecture

Component	Purpose
Route 53 and CloudFront	DNS for srmcuriousbees.in; content delivery with TLS, caching only static assets
Application Load Balancer	Routes /api/* to the API containers and all other paths to the web containers
ECS Fargate (ARM64)	Two web and two API containers in public subnets behind locked-down security groups, avoiding the cost of a NAT gateway
RDS PostgreSQL 17	Managed database in isolated subnets, with encryption, backups and point-in-time recovery
Amazon S3	Private bucket for research files, accessed only through presigned URLs
ECR and SSM Parameter Store	Container image registry; secrets injected at run time, never stored in images or the repository
CloudWatch	Structured logs with 30-day retention and alarms
IAM with GitHub OIDC	Least-privilege roles; the pipeline deploys without any long-lived cloud keys

3.12 Web Application Security Practices

The OWASP Top 10 lists the most critical security risks to web applications (OWASP Foundation, 2021). I used it as a checklist while building CuriousBees. Table 3.11 relates the main risks to the measures taken in the platform.

Table 3.11: Security Risks and Their Mitigation in CuriousBees

Risk (OWASP Top 10:2021)	**Mitigation in CuriousBees**
A01 Broken access control	Every permission is checked on the server by role guards, the ResearchParticipantGuard and workspace membership checks
A02 Cryptographic failures	HTTPS only; S3 bucket policy denies insecure transport; secrets kept out of the code and images
A03 Injection	Parameterised queries through Prisma; strict validation of every request body
A04 Insecure design	Administrator-provisioned accounts; production refuses to start with missing or unsafe configuration
A05 Security misconfiguration	Helmet security headers; CORS allow-list; Swagger disabled in production
A07 Identification and authentication failures	Token verified on every request; allowed email domains; rate-limited OTP sign-in
A08 Software and data integrity failures	Automated CI checks before release; keyless deployment through OIDC; migration drift check
A09 Security logging and monitoring failures	Structured JSON logs with request identifiers; audit log of sensitive administrative actions

3.13 Performance, Scalability and Operational Safety

A platform for an institution must remain responsive when many people use it at once, and must keep running safely when something goes wrong. Table 3.12 lists the main controls built into CuriousBees for this purpose.

Table 3.12: Performance and Operational Safety Controls

Control	**Purpose**
Cursor pagination capped at 50 items	Keeps every list response small, however much data exists
Rate limiting	Protects the API from abuse; 120 requests per minute per client at delivery, refined to per-session limits in October 2026
Database connection limit per instance	Prevents the API from exhausting the database's connections (planned limit of 5 per container)
Database indexes	Fast filtering and sorting of the largest tables

Control	Purpose
PostgreSQL advisory lock for the daily digest	Ensures that scheduled work runs once, even with several API instances
Separate liveness and readiness checks	A brief database interruption does not cause healthy containers to be restarted
Graceful shutdown	Containers finish their requests and close database connections before they stop
Response compression	Smaller responses and faster pages on slow networks

The most important lesson in this area was to measure before adding infrastructure. Message queues and a real-time socket server were added early and later removed because the platform did not need them at its scale. A smaller system with fewer moving parts is cheaper, easier to operate and easier to reason about.

3.14 Documentation, Version Control and Team Practices

Software built by a team must be understandable by people who did not write it. I kept the repository history readable with the Conventional Commits format, in which each message starts with its type, such as feat for a new feature or fix for a correction. Figure 3.11 shows the distribution of the 129 commits made up to delivery: most added features, and a substantial number restructured existing code without changing its behaviour, which reflects the deliberate simplification described earlier.

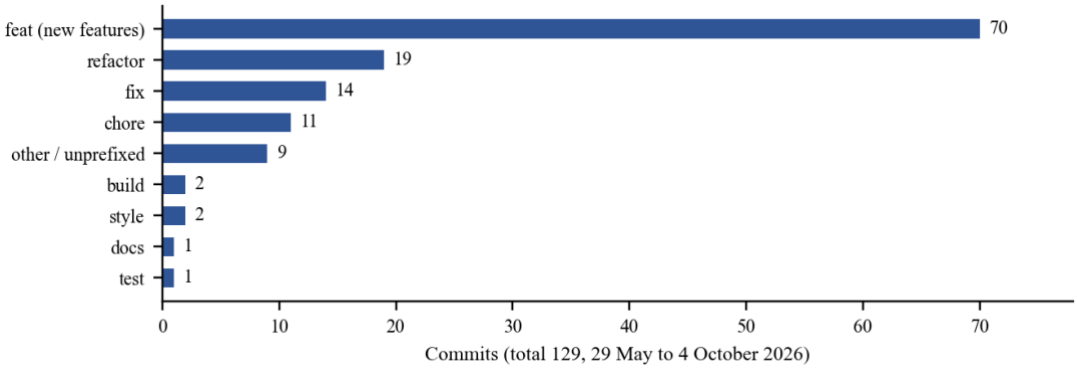


Figure 3.11: Commits by Conventional Commits type up to delivery

Alongside the code, I maintained a documentation portal of 40 documents: architecture and database descriptions, deployment guides for each environment, a security audit, the design system, setup guides for three operating systems, and a release checklist. Each document was written for a reader outside the project, including the ITKM team that will deploy the platform.

4. DAILY/WEEKLY LOG OF ACTIVITIES

The logs in this chapter are based on the version-control history of the delivered repository. During the internship period, from 25 May to 25 August 2026, 111 commits were made on 24 working days; by the delivery on 4 October 2026, the total had reached 129. Figure 4.1 shows how this activity was distributed across the weeks, and Figure 4.2 shows the main phases of the project.

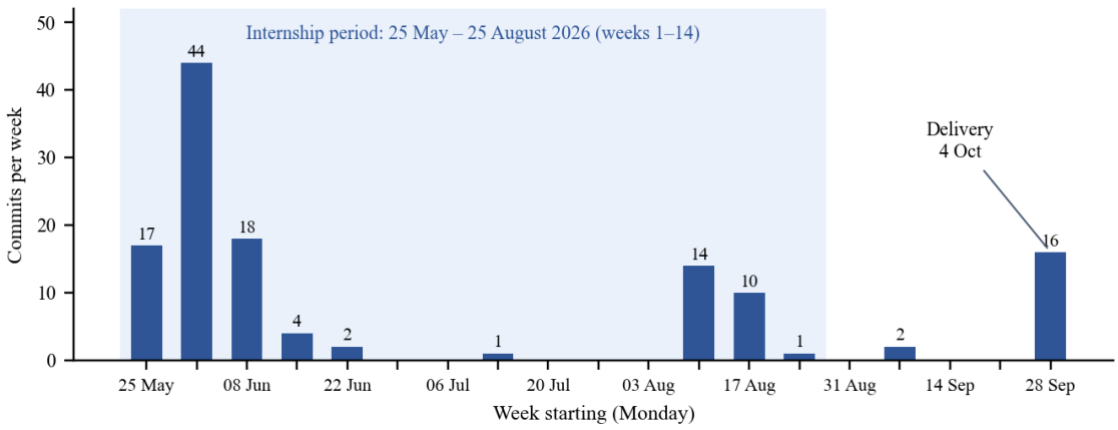


Figure 4.1: Weekly commit activity in the delivered repository (May–October 2026)

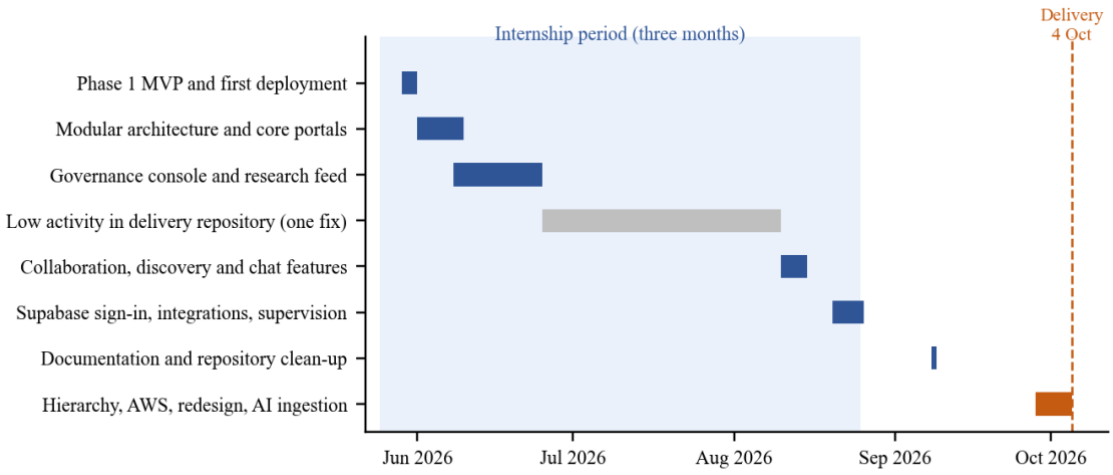


Figure 4.2: Timeline of the main project phases

4.1 Weekly Log

The internship lasted from 25 May 2026 to 25 August 2026. The activities carried out each week are summarised in Table 4.1, and the completion and delivery work that followed is summarised in Table 4.2.

Table 4.1: Weekly Log of Internship Activities

Week	Period	Activities
1	25 – 31 May 2026	Internship commenced on 25 May. Created the monorepo (Next.js web application, NestJS API, shared packages) as the SRM ReCollab Phase 1 MVP; first deployment on Vercel with serverless, Prisma, routing, health-check and CORS fixes; Supabase PostgreSQL schema and seed; Firebase sign-in; first AI event-ingestion prototype and pgvector analytics. (17 commits)
2	01 – 07 Jun 2026	Workspace management with approval workflows; role-based authentication, route protection and portal dashboards; opportunities portal; admin dashboard with moderation and audit logs; modular restructuring of the architecture; security hardening, API documentation, environment validation and indexes; Redis and BullMQ added, audited and removed; authentication moved to Clerk; faculty–department validation; API hardened for Railway. (44 commits)
3	08 – 14 Jun 2026	Clerk sign-in stabilised (two-step sign-in, 2FA, password reset); institute administration panel and route conflict fix; public marketing pages; configurable allowed email domains; admin modules consolidated into one user-management structure; brand palette; scholar portal routing; research feed with discovery, nested comments and thread actions. (18 commits)
4	15 – 21 Jun 2026	Faculty field in the feed schema; chat and post-management modals; improved thread loading and synchronisation; save and comment support. (4 commits)
5	22 – 28 Jun 2026	Public post view with share modal; feed sorting, saved-posts filter and peer connections with notifications. (2 commits)
6 – 11	29 Jun – 09 Aug 2026	One change committed to the delivery repository: the API's network binding corrected for deployment (13 Jul). The modules contributed by team members were developed in separate repositories and integrated later (Section 4.4). (1 commit)
12	10 – 16 Aug 2026	Modular events and opportunities dashboards with role-based access; event editing; Curious Nexus hub; researcher directory with follow and unfollow; scholar onboarding with email alerts and audit logging; multi-level threaded comments with notifications; richer opportunity model; collaboration requests; chat module. (14 commits)
13	17 – 23 Aug 2026	Authentication migrated from Clerk to Supabase; institutional email OTP fallback with rate limiting; integrations module for Google Workspace and Zoom; supervisor request management; scholar reassignment; unified account settings; supervision capacity standardised at six. (10 commits)
14	24 – 25 Aug 2026	Socket-based chat module removed in favour of a simpler design; documentation of research workflows and a capacity analysis; internship period completed on 25 August. (1 commit)

Table 4.2: Completion and Delivery after the Internship Period

Period	Activities	Outcome
08 Sep 2026	README overhaul, repository organisation and removal of legacy names	Clean, documented repository (2 commits)
28 Sep – 04 Oct 2026	Phase 2 canonical institutional hierarchy (v0.4.0) with tests; AWS deployment infrastructure, IAM policies and CI/CD; ResearchParticipantGuard, administrator safety rules and root-administrator protection; platform redesign and design system; single and bulk user provisioning; Google Workspace Chat spaces; workspace creation; email-to-event ingestion with Gemini 2.5 Flash and its tests; final build fix	Integrated, tested, deployment-ready platform delivered on 4 October (16 commits)
October 2026 (after delivery)	Production-readiness hardening: per-session rate limits, a time limit on sign-in verification, dependency security updates, a safe load-testing harness and stronger database safety guards	Better prepared for deployment by ITKM

4.2 Daily Log

The day-wise activities of the internship period are given in Table 4.3. It lists every day on which work was committed to the delivery repository; work done in team members' separate repositories is not shown.

Table 4.3: Daily Log of Internship Activities

Date (2026)	Day	Activity
25 May	Mon	Internship commenced under the Faculty-Guided Summer Internship Programme of the Department of Computer Applications.
29 May	Fri	Created the monorepo as the SRM ReCollab Phase 1 MVP with a light theme and an events AI agent; rebranded to ReCollab; first Vercel deployment with a health-check controller, production CORS, a serverless handler and Prisma binary targets.
30 May	Sat	App-specific Vercel configurations; health check and global prefix refined; Supabase schema and seed script; Supabase client and Prisma direct URL.
31 May	Sun	Firebase Authentication; AI-powered event ingestion prototype; event notification services; analytics, retrieval and search services.
03 Jun	Wed	Workspace management with approval workflows; role-based authentication and route protection; portal dashboard layout and role-specific pages.
04 Jun	Thu	Modular restructuring of the architecture; opportunities portal with faculty submission workflow; admin dashboard with moderation, audit logs and monitoring; authentication guard with automated onboarding.
05 Jun	Fri	Security hardening, API documentation, environment validation and database indexes; removal of dead code and redundant tables.

Date (2026)	Day	Activity
06 Jun	Sat	Centralised environment configuration and a diagnostic "doctor" script; global logging middleware; standardised health checks.
07 Jun	Sun	Authentication migrated to Clerk; faculty–department validation, supervisor approval rules and registration emails; monorepo build orchestration; API hardened for Railway; Redis and BullMQ audited and removed.
08 Jun	Mon	Clerk sign-in flow with two-step sign-in, 2FA and password reset; institute admin panel and route-group conflict fix; public marketing pages; configurable allowed email domains.
09 Jun	Tue	User management consolidated into one component structure; brand colour palette; session synchronisation fixes.
11 Jun	Thu	Scholar portal routing, feed thread details and workspace pages.
12 Jun	Fri	Research feed with discovery endpoints, nested comments and thread actions.
21 Jun	Sun	Faculty field in the feed schema; chat and post-management modals; thread loading, saving and comments.
23 Jun	Tue	Public post view page with a share modal.
24 Jun	Wed	Feed sorting, saved-posts filter and peer connections with notifications.
13 Jul	Mon	API network binding corrected so that the server listens on the default interfaces.
10 Aug	Mon	Events and opportunities dashboards rebuilt as modules with role-based access.
11 Aug	Tue	Event editing and deletion; Curious Nexus hub; researcher directory with follow and unfollow; scholar onboarding with email alerts and audit logging.
12 Aug	Wed	Profile menu and feed navigation; access features across dashboard, events and community; feed components.
13 Aug	Thu	Multi-level threaded comments with notifications; richer opportunity model; collaboration flow and chat interface.
14 Aug	Fri	Chat module with a gateway; deployment documentation.
20 Aug	Thu	Authentication migrated from Clerk to Supabase; email OTP fallback with rate limiting; Google Workspace and Zoom integrations; supervisor request management.
21 Aug	Fri	Scholar reassignment; unified account settings; supervision capacity standardised at six.
25 Aug	Tue	Socket-based chat module removed; research workflow documentation and capacity analysis; internship completed.

4.3 Major Projects and Their Outcomes

The major outcome of the internship was a working, integrated research collaboration platform. The work can be grouped into the following outcomes.

- **A complete platform for three roles:** research scholars, supervisors and administrators each have their own portal, with the permissions of each role enforced by the API.
- **Institution-faithful data model:** campus, faculty and department records anchor every user, supervision relationship, event and opportunity.
- **Secure collaboration:** private workspaces with direct-to-storage file sharing, a research feed and Curious Nexus conversations.
- **AI that saves manual work safely:** institutional event emails become calendar entries, with uncertain cases reviewed by administrators.
- **Deployment readiness:** automated tests, a CI/CD pipeline, container images and a prepared cloud architecture.

Table 4.4 summarises the major modules and the outcome of each at delivery, and Figures 4.3 to 4.10 show the delivered application.

Table 4.4: Major Modules and Their Outcomes

Module	What was built	Outcome at delivery
Authentication and onboarding	Supabase Google sign-in for allowed domains, OTP fallback, administrator-provisioned accounts, onboarding	Institution-only access; no self-registration
Supervision	Requests, approvals, capacity of six, reassignment, email alerts, Supervision Panel	End-to-end supervision workflow
Research record	My Research: thesis topic, stages, milestones, progress reports and publications	Structured progress tracking
Workspaces	Members, files (S3, 50 MB, presigned URLs), milestones, updates, meetings, Chat spaces	Private collaboration spaces
Feed and Curious Nexus	Posts, papers, questions, nested comments, likes, saves, shares, reports, collaboration requests	Institution-wide research discovery
Opportunities and events	Opportunity postings, join requests and an events calendar	One place for positions and events
Governance console	Users, bulk import, institution structure, moderation, announcements, analytics, audit centre	Fully administrable platform

Module	What was built	Outcome at delivery
Email-to-event ingestion	n8n workflow, rule-based filter, Gemini 2.5 Flash, validation, Event Inbox	Events from email without manual entry
DevOps and quality	CI/CD, AWS infrastructure templates, Docker, 270 automated tests	Deployment-ready build for ITKM

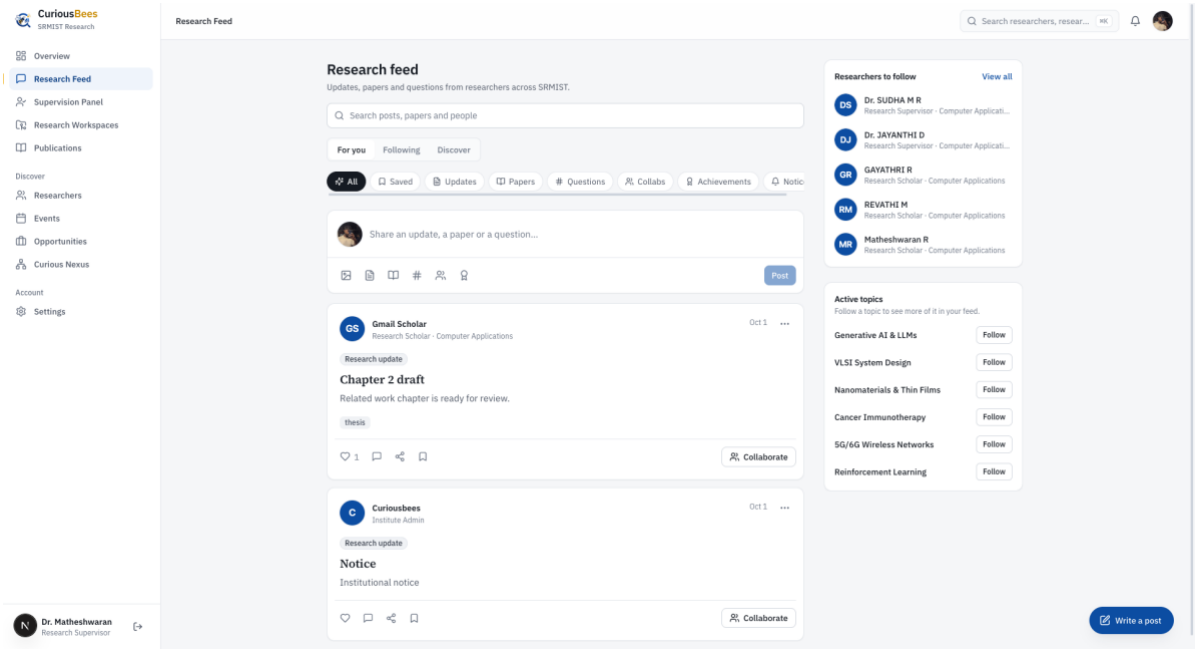


Figure 4.3: Research feed as seen by a research supervisor

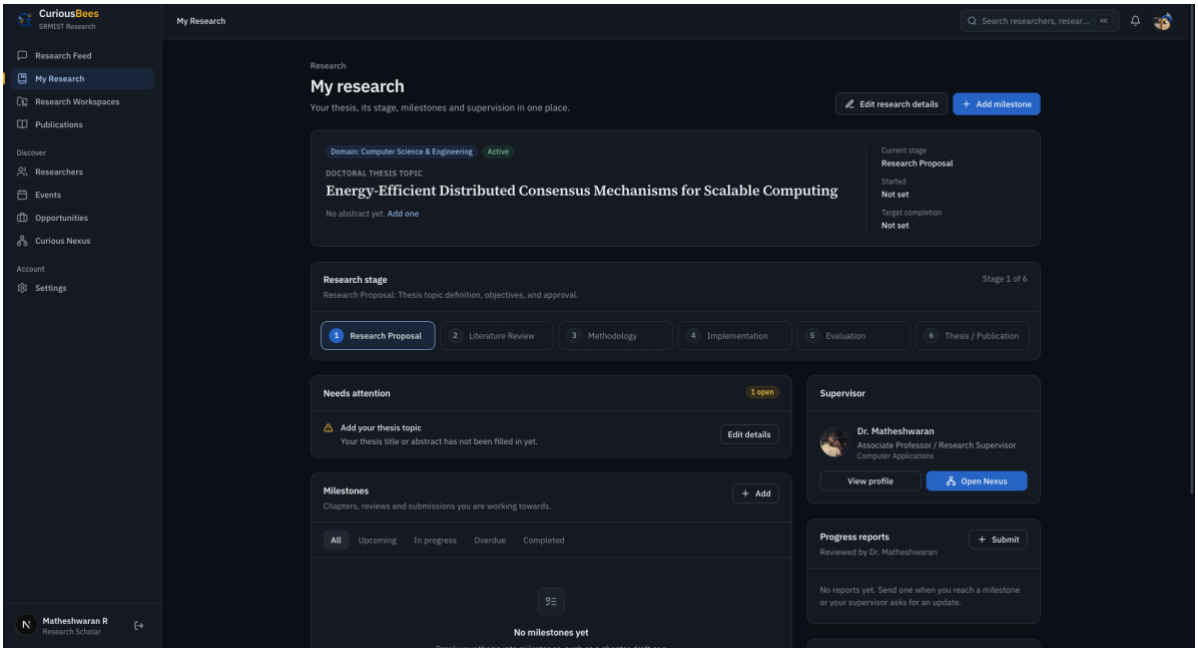


Figure 4.4: My Research page of a research scholar: thesis topic, research stage, supervisor and milestones

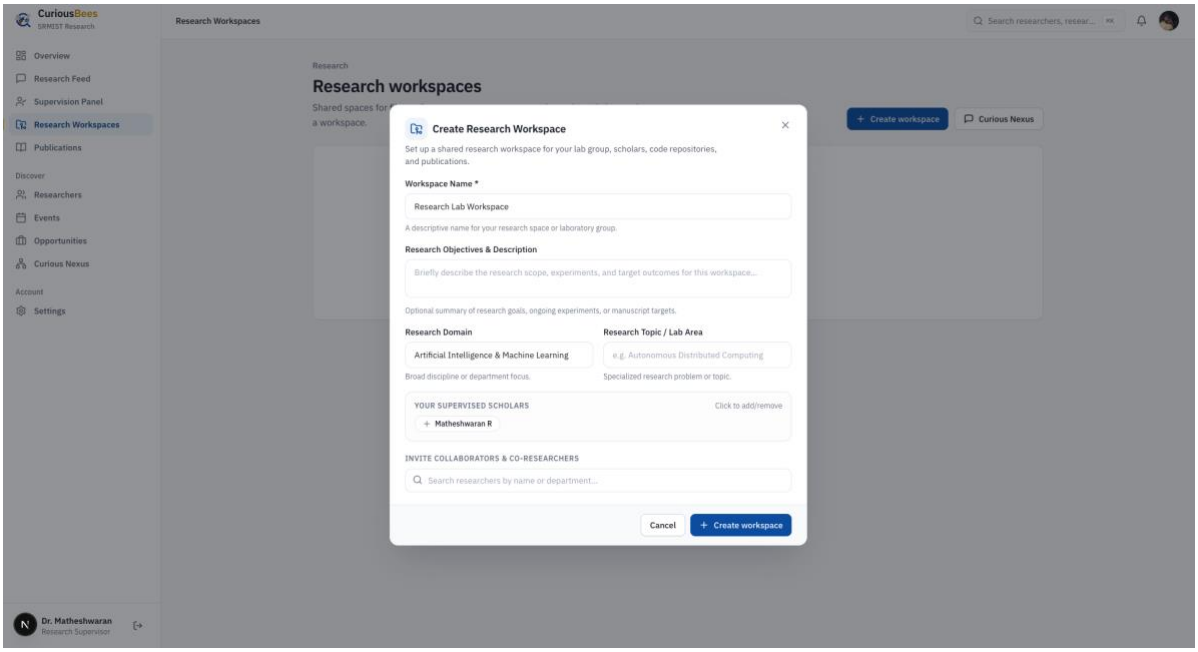


Figure 4.5: Creating a research workspace for a supervisor's scholars

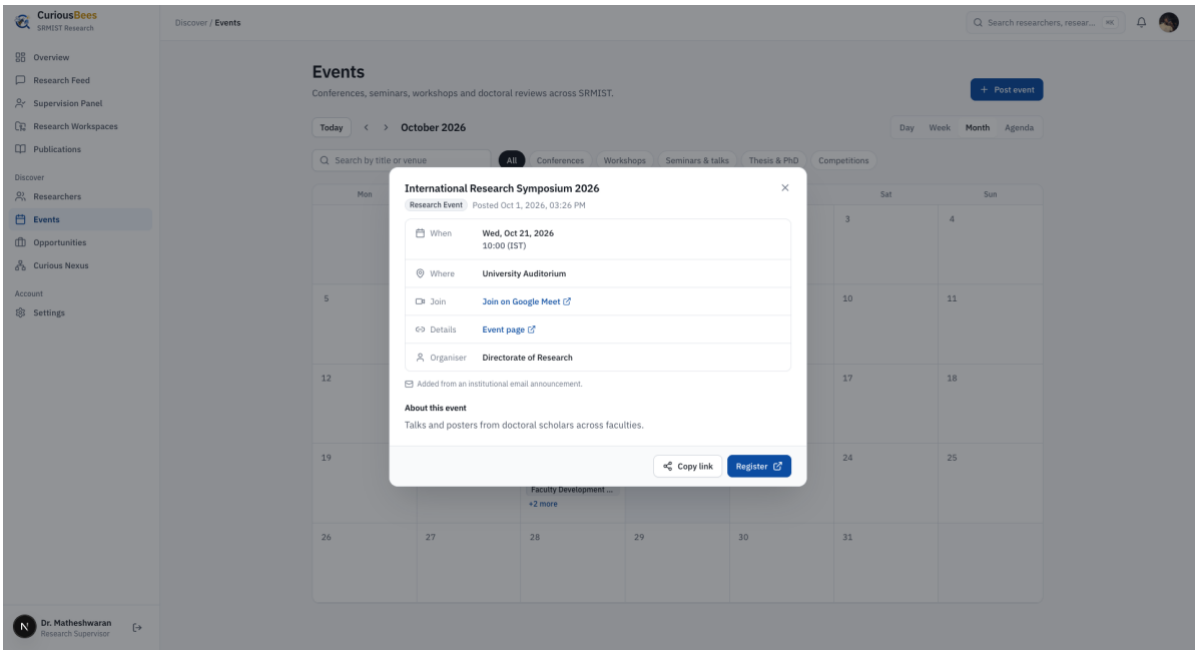


Figure 4.6: Events calendar with an event created from an institutional email announcement

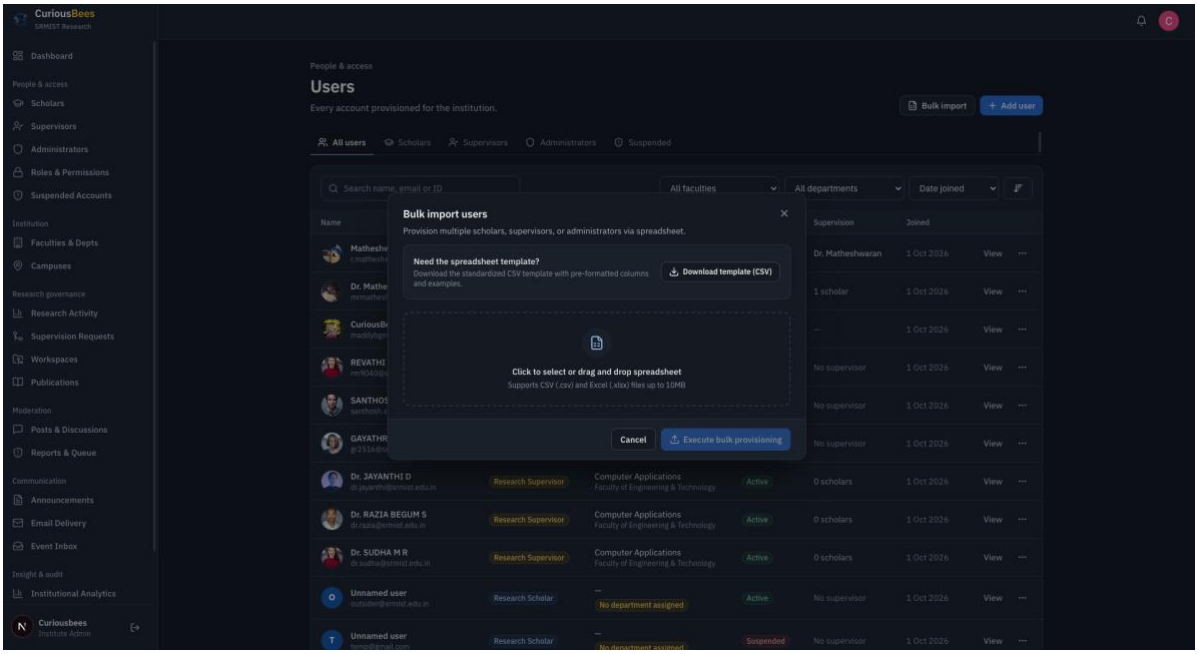


Figure 4.7: Institute administrator's user directory with bulk provisioning from a spreadsheet

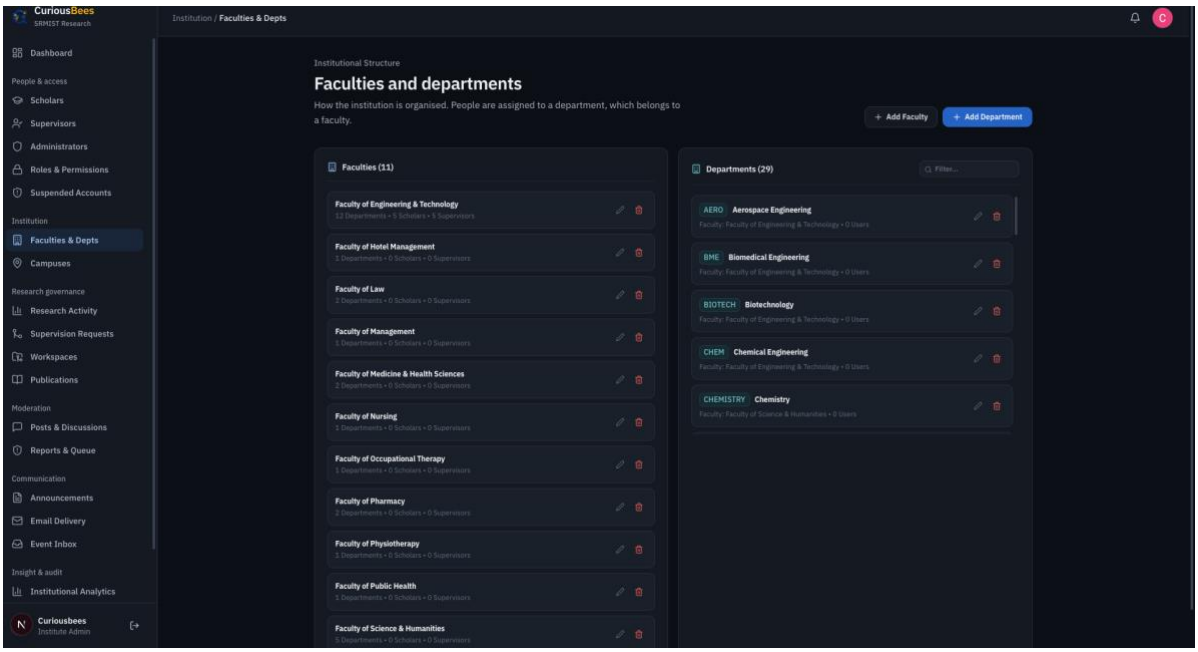


Figure 4.8: Institutional structure managed by the administrator: 11 faculties and 29 departments

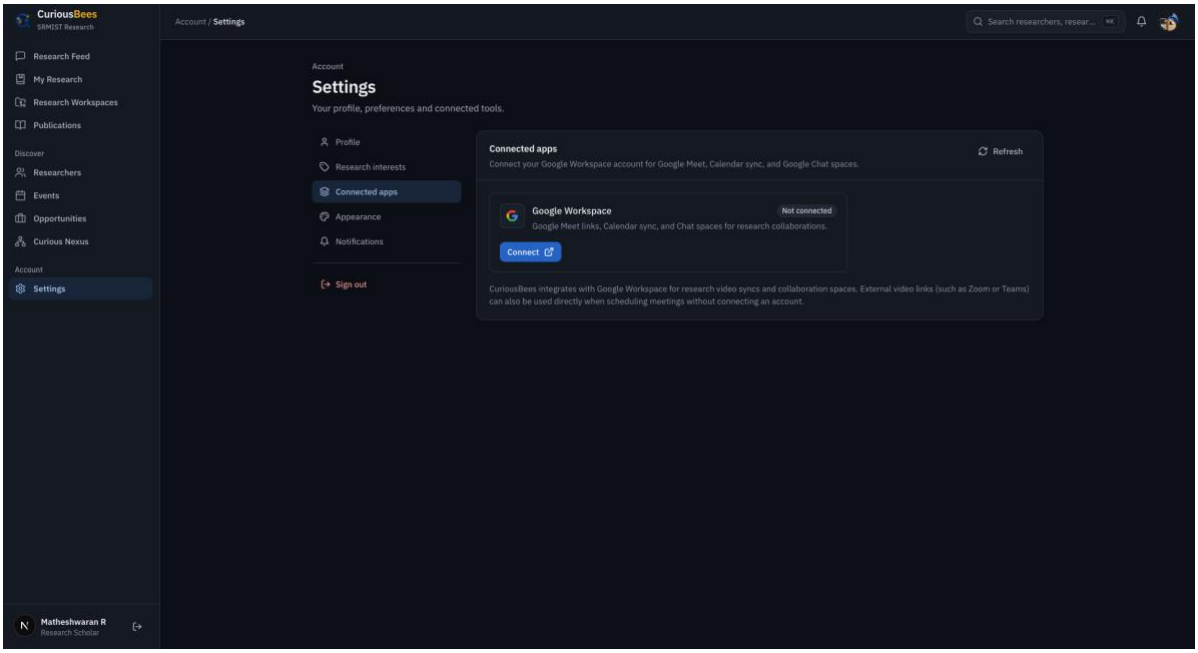


Figure 4.9: Account settings: connecting Google Workspace for meetings and Chat spaces

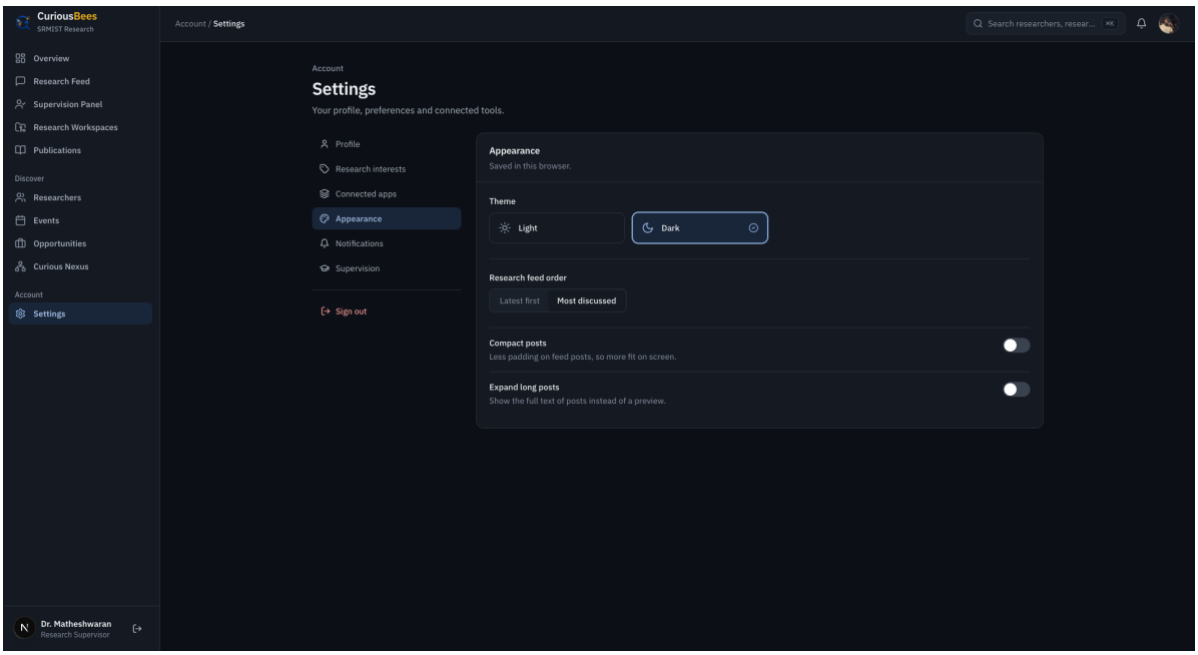


Figure 4.10: Account settings: appearance preferences with light and dark themes

These outcomes are supported by automated evidence. At delivery, all 270 automated tests passed. The integration suite exercised the real HTTP pipeline against a real PostgreSQL database and an S3-compatible store, and the production builds of both applications completed successfully.

4.4 Challenges Faced and How They Were Addressed

Like any real project, the internship had challenges. The main challenges and the way each was addressed are listed in Table 4.5.

Table 4.5: Challenges Faced and Solutions

Challenge	How it was addressed
Deploying a NestJS and Prisma monorepo as serverless functions failed in several ways in the first week.	Over eight deployment commits on 29 and 30 May, I exported a serverless handler, set the Prisma binary targets, added app-specific configurations and a health check, and corrected the API prefix. The experience led to the later choice of long-running containers.
The authentication provider changed twice: Firebase to Clerk in June, and Clerk to Supabase in August.	I settled on a model in which the provider only proves identity, while CuriousBees decides permissions from its own database. This made the final migration manageable and keeps authorization independent of any provider.
A development-mode bypass of authentication and CORS, added for team convenience, would be dangerous in production.	I replaced it with administrator-provisioned accounts, strict production configuration validation that refuses to start when settings are missing, and server-side role guards.
Route conflicts in the App Router and small coding errors broke production builds.	I renamed the conflicting route, fixed each build error, and moved a conditional React hook above the early returns. The CI pipeline now runs linting, type checking and builds on every change.
Redis queues and a socket-based chat server increased the number of services to deploy and monitor.	I removed both after measuring the real need. Scheduled work uses a PostgreSQL advisory lock, and conversations moved to Curious Nexus and Google Workspace Chat spaces.
The schema grew to 54 models, and the early migration history was unsuitable for a clean production database.	I consolidated a production baseline, made every later change a forward-only migration, and added a CI check that the migrations match the schema.
Team modules were built in separate repositories and had to become one product.	I used the shared type and constant packages as the contract, adapted each contribution to the common structure and access rules, and verified the combined system with integration tests.
During one test run, a configuration fallback pointed the integration suite at my local development database, and its demo and test data were lost.	I treated it as a design flaw: tests never read the development configuration, the reset fails closed and runs only against one disposable test database, credentials are removed and outbound network access is blocked. In October, I added a live check of the database's identity before any reset.
A language model can misread dates or invent links, and calling it for every email would be costly.	I placed a rule-based filter before the model, required a strict JSON schema at temperature 0, and validated every answer independently, with human review for uncertain cases.

Overcoming these challenges improved my confidence, my judgement in technical decisions, and my ability to turn a problem into a lasting improvement in the design rather than a one-time fix.

5. SKILLS ACQUIRED

5.1 Technical Skills

- Full-stack development in TypeScript with Next.js, React and NestJS.
- Designing REST APIs with validation, consistent errors, pagination and documentation.
- Relational data modelling, normalisation and safe migrations with PostgreSQL and Prisma.
- Implementing authentication and role-based authorization.
- Secure file handling with private cloud storage and presigned URLs.
- Integrating a large language model into a production workflow responsibly.
- Writing unit and integration tests, and building CI/CD pipelines with containers.
- Planning cloud infrastructure on AWS with least-privilege access.

Table 5.1 relates each of these skills to how it was developed during the internship and where it can be applied.

Table 5.1: Technical Skills and Their Application

Skill	How it was developed	Where it can be applied
Full-stack TypeScript	Building the web application, the API and the shared packages	Any modern web product
API design	Designing 253 endpoints across 40 controllers	Backend services, mobile app backends, integrations
Data modelling	A 54-model schema and the Phase 2 hierarchy migration	Enterprise and institutional systems
Authentication and authorization	Three provider migrations; role guards	Any multi-user application
Cloud storage and security	Presigned URLs, OWASP-based hardening	Systems that handle private documents
Applied AI	Email-to-event pipeline with structured output and validation	Document processing and automation
Testing and CI/CD	270 automated tests and a GitHub Actions pipeline	Professional software teams
Cloud architecture	Prepared AWS design with containers and managed services	DevOps and cloud engineering roles

Beyond individual technologies, I learned to reason about a system as a whole: how a decision in the database schema affects authorization, how an authentication provider affects deployment, and how configuration can make a system safe or unsafe.

5.2 Soft Skills

- **Leadership:** leading eleven students across six technical tracks taught me to set direction, divide a large goal into modules that people can own, and remain accountable for the integrated result.
- **Communication:** I reported progress and decisions to the Faculty Coordinator, explained interfaces and conventions to team members, and wrote documentation for readers outside the project.
- **Conflict resolution:** integration regularly brought competing approaches into conflict. I resolved them by referring to the shared contracts and the institution's requirements rather than to personal preference, and by explaining the reasons for each decision.
- **Time management:** delivering a usable product within the internship, and completing integration before the delivery date, required strict prioritisation and the removal of work that added complexity without value.
- **Ownership and integrity:** when the testing incident occurred, I reported it openly and fixed its cause rather than its symptom.

The completion certificate records commendable learning, strong technical performance and a professional work ethic, qualities that grew from these habits during the three months of the internship.

5.3 New Knowledge and Competencies Developed

The internship gave me knowledge that course work rarely covers. I learned to design for operational safety, with fail-closed checks, forward-only migrations and test isolation from real data. I learned how to integrate a language model into a production workflow with structured output, deterministic pre-filtering, independent validation and human review. I learned to plan cloud infrastructure with least-privilege access and keyless deployment, and to build the governance features that institutions require: administrator-provisioned accounts, protected root administration and audit trails. Finally, I learned to write documentation that allows another team to deploy and operate a system.

5.4 Analysis and Discussion

Looking back, the decisions that served the project best were structural. The monorepo and the shared type packages kept the frontend and backend consistent while many people worked in parallel. Keeping authorization inside the application, independent of the identity provider, allowed the authentication provider to change twice without rewriting the access

rules. The automated tests verified the integration of the team's modules and caught regressions before delivery.

5.4.1 Achievement of objectives

The outcomes of the internship matched the objectives set at the beginning. Table 5.2 relates the objectives to the work through which they were achieved.

Table 5.2: Objectives and How They Were Achieved

Objectives	Achieved through
1–2 Production platform; web architecture	The monorepo, web application and API built from week 1; delivery on 4 October (Sections 3.4–3.6)
3–4 Governance roles; institutional model	Administration console (weeks 2–3); role guards and the Phase 2 hierarchy (September)
5–6 Service integrations; applied AI	Storage, email and meeting integrations (weeks 12–13); email-to-event pipeline (October)
7–9 Security; engineering practice; deployment	Hardening from week 2; tests and CI/CD; Vercel, Railway and the prepared AWS architecture
10–12 Leadership; documentation; real delivery	Planning, review and integration throughout; 40 documents; delivery to the institution for deployment by ITKM

5.4.2 Importance in the field of web development

Professional web teams are expected to design APIs, model data, secure their systems, automate testing and deployment, and increasingly to integrate AI services responsibly. The internship gave me practical experience of each of these areas on one real product.

5.4.3 What I would do differently

The project also showed what I would do differently. The authentication provider should have been chosen earlier, after a short evaluation, rather than changed twice. Infrastructure such as message queues and real-time sockets should be introduced only when a measured need exists. Automated tests and test isolation should be in place from the first week, not the last; the testing incident described in Section 4.4 would not have happened if the destructive test reset had been designed to fail closed from the start.

5.4.4 Limitations and scope for further learning

Production deployment rests with ITKM, so I have not yet operated the platform under real institutional load. Load testing at full scale, production monitoring and incident response remain areas for further learning.

6. CONCLUSION

6.1 Summary of the Internship Experience

The three-month in-house internship at the Department of Computer Applications, SRMIST, from 25 May 2026 to 25 August 2026, was a valuable learning experience. As Team Lead and Full Stack Developer of the eleven-member CuriousBees team, I helped turn an empty repository into a research collaboration and governance platform with role-based portals for research scholars, supervisors and institute administrators. The platform covers supervision, research records, workspaces with private file storage, a research feed, opportunities, events, notifications and a full administrative console, together with an AI-assisted pipeline that turns institutional event emails into calendar entries.

Technically, the work spanned a Next.js and React frontend, a NestJS API with 253 endpoints, a 54-model PostgreSQL schema, integrations with Supabase, Amazon S3, Brevo, Google Workspace and Zoom, an n8n workflow with Gemini 2.5 Flash, 270 automated tests and a CI/CD pipeline with a prepared AWS deployment architecture. As Team Lead, I planned the work, allocated modules, reviewed and integrated the contributions of my team, and delivered the final build on 4 October 2026 for deployment by ITKM. The department recognised this work in the internship completion certificate, which I regard as an encouraging achievement at the start of my professional journey.

6.2 Influence of the Internship on Career Goals

The internship showed me the difference between writing code and delivering a system that an institution can rely on. Taking responsibility for the integrated product, its security and its handover strengthened my interest in full-stack engineering and technical leadership, where design decisions and teamwork matter as much as individual features. It also gave me practical experience of cloud infrastructure, automated testing and applied AI, which I intend to build upon.

In the future, I would like to work in roles such as full-stack developer, cloud and DevOps engineer, or technical lead, where the ability to design complete systems and lead a team is highly valued. I am grateful to the Department of Computer Applications for this opportunity, and I look forward to applying the knowledge gained in my academic projects and my future career.

REFERENCES

Amazon Web Services. (n.d.). *AWS documentation: Amazon S3, Amazon ECS, Amazon RDS and Amazon CloudFront*. https://docs.aws.amazon.com/

Brevo. (n.d.). *Brevo API reference*. https://developers.brevo.com/

Docker Inc. (n.d.). *Docker documentation*. https://docs.docker.com/

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

GitHub. (n.d.). *GitHub Actions documentation*. https://docs.github.com/en/actions

Google AI for Developers. (n.d.). *Gemini API: Structured output*. https://ai.google.dev/gemini-api/docs/structured-output

Google for Developers. (n.d.). *Google Workspace developer documentation*. https://developers.google.com/workspace

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Meta Open Source. (n.d.). *React documentation*. https://react.dev/

Microsoft. (n.d.). *TypeScript documentation*. https://www.typescriptlang.org/docs/

n8n GmbH. (n.d.). *n8n documentation*. https://docs.n8n.io/

NestJS. (n.d.). *NestJS documentation*. https://docs.nestjs.com/

OpenJS Foundation. (n.d.). *Jest documentation*. https://jestjs.io/docs/getting-started

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Prisma Data. (n.d.). *Prisma ORM documentation*. https://www.prisma.io/docs

SRM Institute of Science and Technology. (n.d.). *Department of Computer Applications*. https://www.srmist.edu.in/department/department-of-computer-applications/

SRM Institute of Science and Technology. (2023). *Strategic plan 2023 (Plan period 2024–2033)*. https://webstor.srmist.edu.in/web_assets/downloads/2023/srmist-strategic-plan.pdf

Supabase. (n.d.). *Supabase Auth documentation*. https://supabase.com/docs/guides/auth

Tailwind Labs. (n.d.). *Tailwind CSS documentation*. https://tailwindcss.com/docs

Vercel. (n.d.). *Next.js documentation*. https://nextjs.org/docs

Zoom Video Communications. (n.d.). *Zoom developer documentation*. https://developers.zoom.us/docs/

APPENDIX A: GLOSSARY OF TERMS

Table A.1 explains the technical terms and abbreviations used in this report.

Table A.1: Glossary of Terms

Term	Meaning
API	Application Programming Interface; the set of HTTP endpoints through which the web application talks to the server.
REST	Representational State Transfer; an architectural style for APIs based on resources and HTTP methods.
Monorepo	A single repository that holds several related applications and packages.
ORM	Object-Relational Mapper; a library that maps database tables to typed objects in code (Prisma in this project).
Migration	A versioned script that changes the structure of a database.
Normalisation	Organising data so that each fact is stored once and referred to elsewhere by a key.
RBAC	Role-Based Access Control; granting permissions to roles rather than to individuals.
OTP	One-Time Password; a short code sent to a user's email for a single sign-in.
Presigned URL	A short-lived, server-signed link that allows direct access to one object in private storage.
CI/CD	Continuous Integration and Continuous Delivery; automated building, testing and release of software.
IAM	Identity and Access Management; the AWS system of users, roles and permissions.
OIDC	OpenID Connect; used here to let the CI pipeline obtain short-lived cloud credentials without stored keys.
LLM	Large Language Model; an AI model trained on text that can read and generate language.
Structured output	An LLM answer constrained to a defined schema, such as JSON with fixed fields.
Rate limiting	Restricting how many requests a client can make in a period of time.
Advisory lock	A PostgreSQL lock used by applications to ensure that only one process runs a task at a time.
CORS	Cross-Origin Resource Sharing; browser rules that control which websites may call an API.
OWASP	Open Worldwide Application Security Project; publisher of the OWASP Top 10 security risks.
MCA	Master of Computer Applications.

Term	Meaning
NAAC	National Assessment and Accreditation Council.
UGC	University Grants Commission.

APPENDIX B: PROJECT STATISTICS AT DELIVERY

The figures in Table B.1 were measured from the delivered repository at its final commit of 4 October 2026, and from the version-control history.

Table B.1: Project Statistics at Delivery

Measure	Value
Commits to delivery (29 May – 4 October 2026)	129, on 31 days
Commits during the internship period (25 May – 25 August 2026)	111, on 24 days
Web application page routes	75
API controllers and HTTP endpoints	40 controllers, 253 endpoints
Database schema	54 models, 21 enumerations
TypeScript source lines (including tests)	About 56,700 (web 32,852; API 22,992; shared packages 881)
Automated tests at delivery	270 (172 unit, 61 integration, 34 workflow, 3 web)
Documentation files	40
Team size	11 students in six technical tracks
Final delivery	4 October 2026